

CustomerSegment AI

Customer RFM Clustering & Persona Intelligence Platform

Technology Stack: Python · Scikit-Learn · Pandas · Streamlit · Plotly · SQLite · Redis · DVC
Build Phases: 4 phases | ML Components: 8+ | Cluster Tiers: 3–5 personas | Silhouette Target: 0.50+

Production-ready. Portfolio-grade. Stakeholder-approved.

CustomerSegment AI

Customer RFM Clustering & Persona Intelligence Platform

Technology Stack: Python · Scikit-Learn · Pandas · Streamlit · Plotly · SQLite · Redis · DVC

Build Phases: 4 phases | **ML Components:** 8+ | **Cluster Tiers:** 3–5 personas | **Silhouette Target:** 0.50+

CustomerSegment AI is a production-grade clustering platform that transforms raw transaction data into actionable customer segments. Using RFM feature engineering, KMeans clustering with silhouette optimization, and automated persona generation, it assigns every customer to a business-meaningful cluster (e.g., "High-Value Frequent Buyers," "At-Risk Churners"), explains cluster characteristics via feature importance, and deploys as a Streamlit analytics dashboard with an optional FastAPI backend for real-time scoring.

TABLE OF CONTENTS

#	SECTION	PAGE
1	Executive Summary & Business Case	2
2	System Architecture Overview	3
3	Technology Stack & Rationale	4
4	Data Strategy — RFM Feature Engineering	5
5	ML Engine — KMeans + Silhouette + Persona Generation	6
6	Cluster Framework (3–5 Persona Tiers)	7
7	Phase-by-Phase Build Plan (4 Phases)	8
8	Streamlit Dashboard Design	9
9	Prompt Engineering Library (51 Prompts)	10
10	Evaluation & Business Metrics Framework	11
11	Risk Register & Demo Strategy	12
12	Timeline, Milestones & Pricing	13

1	Executive Summary & Business Case	2
2	System Architecture Overview	3
3	Technology Stack & Rationale	4
4	Data Strategy — RFM Feature Engineering	5
5	ML Engine — KMeans + Silhouette + Persona Generation	6
6	Cluster Framework (3–5 Persona Tiers)	7
7	Phase-by-Phase Build Plan (4 Phases)	8
8	Streamlit Dashboard Design	9
9	Prompt Engineering Library (51 Prompts)	10

10	Evaluation & Business Metrics Framework	15
11	Risk Register & Demo Strategy	16
12	Timeline, Milestones & Pricing	17

1	Executive Summary & Business Case	2
2	System Architecture Overview	3
3	Technology Stack & Rationale	4
4	Data Strategy — RFM Feature Engineering	5
5	ML Engine — KMeans + Silhouette + Persona Generation	6
6	Cluster Framework (3–5 Persona Tiers)	7
7	Phase-by-Phase Build Plan (4 Phases)	8
8	Streamlit Dashboard Design	9
9	Prompt Engineering Library (5 Prompts)	10
10	Evaluation & Business Metrics Framework	11
11	Risk Register & Demo Strategy	12
12	Timeline, Milestones & Pricing	13

2	System Architecture Overview	3
3	Technology Stack & Rationale	4
4	Data Strategy — RFM Feature Engineering	5
5	ML Engine — KMeans + Silhouette + Persona Generation	6
6	Cluster Framework (3–5 Persona Tiers)	7
7	Phase-by-Phase Build Plan (4 Phases)	8
8	Streamlit Dashboard Design	9
9	Prompt Engineering Library (5 Prompts)	10
10	Evaluation & Business Metrics Framework	11
11	Risk Register & Demo Strategy	12
12	Timeline, Milestones & Pricing	13

3	Technology Stack & Rationale	4
4	Data Strategy — RFM Feature Engineering	5
5	ML Engine — KMeans + Silhouette + Persona Generation	6
6	Cluster Framework (3–5 Persona Tiers)	7
7	Phase-by-Phase Build Plan (4 Phases)	8
8	Streamlit Dashboard Design	9
9	Prompt Engineering Library (5 Prompts)	10
10	Evaluation & Business Metrics Framework	11

11	Risk Register & Demo Strategy
12	Timeline, Milestones & Pricing

4	Data Strategy — RFM Feature Engineering
5	ML Engine — KMeans + Silhouette + Persona Generation
6	Cluster Framework (3–5 Persona Tiers)
7	Phase-by-Phase Build Plan (4 Phases)
8	Streamlit Dashboard Design
9	Prompt Engineering Library (5 Prompts)
10	Evaluation & Business Metrics Framework
11	Risk Register & Demo Strategy
12	Timeline, Milestones & Pricing

5	ML Engine — KMeans + Silhouette + Persona Generation
6	Cluster Framework (3–5 Persona Tiers)
7	Phase-by-Phase Build Plan (4 Phases)
8	Streamlit Dashboard Design
9	Prompt Engineering Library (5 Prompts)
10	Evaluation & Business Metrics Framework
11	Risk Register & Demo Strategy
12	Timeline, Milestones & Pricing

6	Cluster Framework (3–5 Persona Tiers)
7	Phase-by-Phase Build Plan (4 Phases)
8	Streamlit Dashboard Design
9	Prompt Engineering Library (5 Prompts)
10	Evaluation & Business Metrics Framework
11	Risk Register & Demo Strategy
12	Timeline, Milestones & Pricing

7	Phase-by-Phase Build Plan (4 Phases)
8	Streamlit Dashboard Design
9	Prompt Engineering Library (5 Prompts)
10	Evaluation & Business Metrics Framework
11	Risk Register & Demo Strategy
12	Timeline, Milestones & Pricing

8	Streamlit Dashboard	Design
9	Prompt Engineering Library (5 Prompts)	
10	Evaluation & Business Metrics Framework	
11	Risk Register & Demo Strategy	
12	Timeline, Milestones & Pricing	

9	Prompt Engineering Library (5 Prompts)	
10	Evaluation & Business Metrics Framework	
11	Risk Register & Demo Strategy	
12	Timeline, Milestones & Pricing	

10	Evaluation & Business Metrics Framework	
11	Risk Register & Demo Strategy	
12	Timeline, Milestones & Pricing	

01 · EXECUTIVE SUMMARY & BUSINESS CASE

The Customer Segmentation Problem That CustomerSegment AI Solves

Marketing teams waste budget treating all customers identically. Without ML-driven segmentation, campaigns lack precision: acquisition budgets go to low-value prospects, retention budgets don't reach churners soon enough, and upsell campaigns miss high-value accounts. CustomerSegment AI changes this fundamentally: every customer receives an RFM (Recency, Frequency, Monetary) profile, is assigned to a cluster ("Platinum," "Silver," "At-Risk," etc.), and the system explains exactly why — which RFM dimensions drove the assignment — in plain English. Marketing managers can see their customer base composition at a glance and tailor spend per segment.

Project Identity

Attribute	Value
Project Name	CustomerSegment AI
Category	Unsupervised ML · Customer Analytics · Marketing Operations
ML Paradigm	Unsupervised Clustering (KMeans) + Silhouette Validation + Rule-Based
Frontend	Streamlit 1.x (Python-native, rapid prototyping)
Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)

ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–12 features)
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of squares) ≤ 0.50

Project Name	CustomerSegmentation AI
Category	Unsupervised ML · Customer Analytics · Marketing Operations
ML Paradigm	Unsupervised Clustering (KMeans) + Silhouette Validation + Rule-Based Segmentation
Frontend	Streamlit 1.x (Python-native, rapid prototyping)
Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–12 features)
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of squares) ≤ 0.50

Project Name	CustomerSegmentation AI
Category	Unsupervised ML · Customer Analytics · Marketing Operations
ML Paradigm	Unsupervised Clustering (KMeans) + Silhouette Validation + Rule-Based Segmentation
Frontend	Streamlit 1.x (Python-native, rapid prototyping)
Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–12 features)
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of squares) ≤ 0.50

Category	Unsupervised ML · Customer Analytics · Marketing Operations
ML Paradigm	Unsupervised Clustering (KMeans) + Silhouette Validation + Rule-Based Segmentation
Frontend	Streamlit 1.x (Python-native, rapid prototyping)
Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)

Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of s

ML Paradigm	Unsupervised Clustering (KMeans) + Silhouette Validation +
Frontend	Streamlit 1.x (Python-native, rapid prototyping)
Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of s

Frontend	Streamlit 1.x (Python-native, rapid prototyping)
Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of s

Backend (Optional)	FastAPI 0.x (async scoring API)
Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of s

Database	Supabase / SQLite (transaction data + cluster assignments)
Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of s

Feature Store	Redis (cached customer profiles; TTL 24h)
ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–12 features)
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of squares)

ML Tracking	MLflow 2.x (experiment tracking, model registry)
Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–12 features)
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of squares)

Deployment	Railway.app (Streamlit) or Hugging Face Spaces (free tier)
Dataset	UCI Online Retail or Synthetic E-Commerce (50k–100k transactions, 8–12 features)
Business KPI	Silhouette Score ≥ 0.50 · Intra-Cluster Cohesion (within-cluster sum of squares)

Why Clients & Recruiters Will Buy / Hire For This

- RFM feature engineering is non-obvious** — most portfolios skip RFM; showing median recency, frequency, monetary per customer + normalization/scaling earns recruiter credibility.
- Silhouette optimization** — rare to see actual hyperparameter tuning (k, init, max_iter) for unsupervised models; demonstrates rigor beyond "run KMeans once."
- Persona generation from cluster statistics** — translating cluster centroids and variance into business-readable personas ("High-Value Frequent," "One-Time Buyer") is a valuable skill.
- Streamlit dashboard** — modern, rapid-deploy alternative to Dash; shows frontend agility.
- Real-time batch recomputation** — assigned customers to clusters weekly; recompute if RFM drifts.
- Quoted business impact** — "customers in Platinum segment have 3.2× higher lifetime value" resonates with stakeholders.

02 · SYSTEM ARCHITECTURE OVERVIEW

Batch segmentation pipeline with weekly recompute and optional real-time scoring API.

Architecture Layers

Layer	Component	Technology	Responsibility
Presentation	Analytics Dashboard	Streamlit 1.x + Plotly	Cluster distribution, persona profiles, member counts
Presentation	Cluster Inspector	Streamlit tabs	Centroid stats, silhouette per cluster, feature correlations

Backend API	Scoring Service (optional)	FastAPI 0.x	Real-time customer → cluster assignment
ML Core	Segmentation Engine	Scikit-Learn KMeans	Feature vector → cluster label
ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignment
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

Presentation	Analytics Dashboard	Streamlit 1.x + Plotly	Cluster distribution, persona profiles, narrative
Presentation	Cluster Inspector	Streamlit tabs	Centroid stats, silhouette per cluster, feature co
Backend API	Scoring Service (optional)	FastAPI 0.x	Real-time customer → cluster assignment
ML Core	Segmentation Engine	Scikit-Learn KMeans	Feature vector → cluster label
ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignment
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > th

Presentation	Analytics Dashboard	Streamlit 1.x + Plotly	Cluster distribution, persona profiles, narrative
Presentation	Cluster Inspector	Streamlit tabs	Centroid stats, silhouette per cluster, feature co
Backend API	Scoring Service (optional)	FastAPI 0.x	Real-time customer → cluster assignment
ML Core	Segmentation Engine	Scikit-Learn KMeans	Feature vector → cluster label
ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignment
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > th

Presentation	Cluster Inspector	Streamlit tabs	Centroid stats, silhouette per cluster, fe
Backend API	Scoring Service (optional)	FastAPI 0.x	Real-time customer → cluster assignment
ML Core	Segmentation Engine	Scikit-Learn KMeans	Feature vector → cluster label

ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

Backend API	Scoring Service (optional)	Frontend API 0.x	Real-time customer insights
ML Core	Segmentation Engine	Scikit-Learn KMeans	Feature vector → cluster label
ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

ML Core	Segmentation Engine	Scikit-Learn KMeans	Feature vector → cluster label
ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

ML Core	Silhouette Analyzer	scikit-learn metrics	Cluster cohesion validation & optimal k selection
ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

ML Core	Persona Generator	Custom class + Jinja2	Cluster stats → narrative business descriptions
Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d

Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

Feature Store	Hot Profiles	Redis Hash per customer	Latest RFM for this week; TTL 7d
Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

Feature Store	Cold History	SQLite / Parquet	Full RFM history; training data archive
Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

Data Pipeline	RFM Calculator	Pandas groupby + agg	Transactions → R, F, M per customer
ML Tracking	Experiment Logs	MLflow 2.x	Silhouette scores, centroids, cluster assignments
Monitoring	Drift Detector	Custom; distribution tests	Check RFM distribution weekly; alert if shift > threshold

Weekly Segmentation Flow

- Data Ingest** (Sundays 23:00 UTC): Pull all transactions from Supabase for prior 365 days.
- RFM Calculation** (Pandas groupby): Compute per-customer Recency (days since last), Frequency (count), Monetary (sum).
- Scaling**: StandardScaler on R, F, M (zero mean, unit variance).
- KMeans Fit**: ``KMeans(n_clusters=optimal_k, init='k-means++', max_iter=300, random_state=42)`` on R, F, M.
- Silhouette Scoring**: ``silhouette_samples()`` per point; compute `silhouette_score` (global). Log to MLflow.
- Cluster Stats Computation**: Mean R, F, M per cluster; variance; count; percentiles.
- Persona Generation**: Template-fill narrative for each cluster (e.g., "Platinum: High Frequency (85 purchases/yr), High Monetary (\$12,500 LTV), Recent (15 days).").
- Results Write**: SQLite ``customer_clusters`` table (customer_id, cluster, centroid_r, centroid_f, centroid_m, persona_name).
- Redis Cache**: HSET `customer:{id} → {rfm_r, rfm_f, rfm_m, cluster, persona}`.
- MLflow Artifact**: Save cluster centroids (numpy), silhouette scores (numpy), persona descriptions (JSON).
- Streamlit Load**: Dashboard reads latest from SQLite + Redis; visualizes cluster distribution, member explorer.

03 · TECHNOLOGY STACK & RATIONALE

Every tool selected for clarity, modern relevance, and portfolio differentiation.

Domain	Tool	Ver	Rationale vs Alternative
Data Processing	Pandas	2.x	Standard EDA; groupby/agg for RFM; no Polars
Feature Scaling	scikit-learn	1.5	StandardScaler + MinMaxScaler options; reproducible
Clustering	scikit-learn KMeans	1.5	Industry standard; `init='k-means++'` avoids poor
Silhouette Analysis	scikit-learn metrics	1.5	silhouette_score, silhouette_samples; validate c
Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, max_iter;
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Clau
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Data Processing	Pandas	2.x	Standard EDA; groupby/agg for RFM; n
Feature Scaling	scikit-learn	1.5	StandardScaler + MinMaxScaler options; repro
Clustering	scikit-learn KMeans	1.5	Industry standard; `init='k-means++'` avoids po
Silhouette Analysis	scikit-learn metrics	1.5	silhouette_score, silhouette_samples; validate c
Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, max_iter;
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Clau
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight

Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Data Processing	Pandas	2.x	Standard EDA; groupby/agg for RFM
Feature Scaling	scikit-learn	1.5	StandardScaler + MinMaxScaler options; repro
Clustering	scikit-learn KMeans	1.5	Industry standard; `init='k-means++'` avoids po
Silhouette Analysis	scikit-learn metrics	1.5	silhouette_score, silhouette_samples; validate c
Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, max_iter;
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Cla
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Feature Scaling	scikit-learn	1.5	StandardScaler + MinMaxScaler option
Clustering	scikit-learn KMeans	1.5	Industry standard; `init='k-means++'` avoids po
Silhouette Analysis	scikit-learn metrics	1.5	silhouette_score, silhouette_samples; validate c
Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, max_iter;
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Cla
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit

CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Clustering	scikit-learn KMeans	1.5	Industry standard;
Silhouette Analysis	scikit-learn metrics	1.5	silhouette_score, silhouette_samples; validate c
Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, max_iter;
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Cla
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Silhouette Analysis	scikit-learn metrics	1.5	silhouette_score, silhouette_samples;
Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, max_iter;
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Cla
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Hyperparameter Tuning	Optuna or loop	3.x	Bayesian search over $k \in [2,10]$, init, m
Visualization	Plotly 5.x	5.x	Interactive cluster plots, silhouette bars; more p
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove

Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Clau
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Visualization	Plotly 5.x	5.x	Interactive cluster
Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/CSS ove
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Clau
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Dashboard	Streamlit	1.x	Python-native, rapid deploy; no HTML/C
Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple); or Clau
Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Text Generation (Personas)	Jinja2 templates	3.x	Rule-based persona templates (simple)
---------------------------------------	------------------	-----	---------------------------------------

Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params; model re
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Experiment Tracking	MLflow	2.x	Log silhouette, centroids, run params;
Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQLite = lo
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Database (Transactional)	Supabase / SQLite	–	Supabase = hosted Postgres (dev); SQ
Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can use SQLite
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Cache Layer	Redis 7.x	7.x	Customer RFM cache; optional (can us
Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible pipelines
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Data Versioning	DVC 3.x	3.x	Track parquet datasets; reproducible p
Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; lightweight
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Batch Scheduler	APScheduler	3.x	Weekly RFM recalc + KMeans refit; ligh
Optional API	FastAPI	0.x	If real-time scoring needed; async; OpenAPI au
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Optional API	FastAPI	0.x	If real-time scoring needed; async; Ope
Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free + no credit
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

Deployment (Streamlit)	Railway / HF Spaces	–	Railway = simple; HF Spaces = free +
CI/CD	GitHub Actions	–	Lint, test, deploy on push
Logging	Logfire / Python logging	–	Structured logs; drift alerts via email/Slack

04 · DATA STRATEGY — RFM FEATURE ENGINEERING

From raw transactions to a 3-feature ML-ready RFM matrix.

Recommended Datasets

Option A: UCI Online Retail (Real E-Commerce Data) — Recommended

- Source:** <https://archive.ics.uci.edu/dataset/352/online+retail>
- Records:** 500k transactions; 5,000 unique customers
- Columns:** InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice, CustomerID, Country
- Preprocessing needed:** Remove returns (Quantity < 0), consolidate by InvoiceDate + CustomerID
- RFM window:** Last 365 days from reference date (2011-12-09)

Option B: Synthetic E-Commerce Data — Good for Demo

- Generated via Faker:** Transactions with CustomerID, OrderDate, OrderAmount, Product
- Records:** 50k transactions; 5k customers; realistic distribution (Pareto: 20% of customers = 80% revenue)
- Advantage:** Clean, no missing values, reproducible seed

Option C: Supermarket Customer Purchase (UCI)

- Source:** <https://archive.ics.uci.edu/dataset/292/supermarket+customer+purchase+data>
- Records:** 5,000 transactions; 1,000 customers; includes product categories
- Simpler:** Smaller scale; good for MVP demo

Recommendation for portfolio: Use **Option A (UCI Online Retail)** — it's real, public, and widely recognized. Show end-to-end workflow on >5k customers.

RFM Calculation — Exact Methodology

Reference Date: Latest date in dataset (e.g., 2011-12-09 for UCI Online Retail).

Formula per customer:

```
Recency (R)    = Days between reference date and customer's LAST transaction
Frequency (F)  = Count of transactions (invoices) in 365-day window
Monetary (M)   = Sum of (Quantity × UnitPrice) in 365-day window
```

Recency (R) = Days between reference date and customer's LAST transaction

Frequency (F) = Count of transactions (invoices) in 365-day window

Monetary (M) = Sum of (Quantity × UnitPrice) in 365-day window

****Calculation code outline:****

Calculation code outline:

```
# Group by CustomerID
rf = df.groupby('CustomerID').agg({
    'InvoiceDate': ['max', 'count'],
    'TotalAmount': 'sum'
}).reset_index()

# Flatten columns
rf.columns = ['CustomerID', 'LastPurchase', 'Frequency', 'Monetary']

# Compute Recency (days)
reference_date = df['InvoiceDate'].max()
rf['Recency'] = (reference_date - rf['LastPurchase']).dt.days

# Select final features
rfm = rf[['CustomerID', 'Recency', 'Frequency', 'Monetary']].copy()
rfm = rfm[(rfm['Frequency'] > 0) & (rfm['Monetary'] > 0)] # Remove inactive
```

```
print(f"Customer count: {len(rfm)}")
print(f"RFM stats:\n{rfm[['Recency', 'Frequency', 'Monetary']].describe()}")
```

Group by CustomerID

```
rf = df.groupby('CustomerID').agg({
    'InvoiceDate': ['max', 'count'],
    'TotalAmount': 'sum'
}).reset_index()
```

Flatten columns

```
rf.columns = ['CustomerID', 'LastPurchase', 'Frequency', 'Monetary']
```

Compute Recency (days)

```
reference_date = df['InvoiceDate'].max()
rf['Recency'] = (reference_date - rf['LastPurchase']).dt.days
```

Select final features

```
rfm = rf[['CustomerID', 'Recency', 'Frequency', 'Monetary']].copy()
rfm = rfm[(rfm['Frequency'] > 0) & (rfm['Monetary'] > 0)] # Remove inactive

print(f"Customer count: {len(rfm)}")
print(f"RFM stats:\n{rfm[['Recency', 'Frequency', 'Monetary']].describe()}")
```

Feature Scaling Strategy

Feature	Scaling Method	Rationale	Output Range
Recency	StandardScaler	Lower R = more recent = better (inverse); mean=0, std=1	■ (typically -2 to +2)
Frequency	StandardScaler	Higher F = more purchases; mean=0, std=1	■
Monetary	StandardScaler	Higher M = higher spend; mean=0, std=1	■

Code:

Feature Scaling Strategy

Feature	Scaling Method	Rationale	Output Range
Recency	StandardScaler	Lower R = more recent purchases (typically 1-2 years)	-1 to 1; mean=0, std=1
Frequency	StandardScaler	Higher F = more purchases	-1 to 1; mean=0, std=1
Monetary	StandardScaler	Higher M = higher spend	-1 to 1; mean=0, std=1

Recency	StandardScaler	Lower R = more recent purchases (typically 1-2 years)	-1 to 1; mean=0, std=1
Frequency	StandardScaler	Higher F = more purchases	-1 to 1; mean=0, std=1
Monetary	StandardScaler	Higher M = higher spend	-1 to 1; mean=0, std=1

Recency	StandardScaler	Lower R = more recent purchases (typically 1-2 years)	-1 to 1; mean=0, std=1
Frequency	StandardScaler	Higher F = more purchases	-1 to 1; mean=0, std=1
Monetary	StandardScaler	Higher M = higher spend	-1 to 1; mean=0, std=1

Code:

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
# rfm_scaled shape: (n_customers, 3)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

rfm_scaled = scaler.fit_transform(rfm[['Recency', 'Frequency', 'Monetary']])
```

rfm_scaled shape: (n_customers, 3)

Dataset Validation Checklist

- ✓ No negative Frequency or Monetary
- ✓ Recency > 0 (customer made at least 1 purchase)
- ✓ No null values in R, F, M
- ✓ Mean Frequency > 1 (customers have repeat purchases, not one-time only)
- ✓ Outliers (top 1% Monetary) identified; decide: keep or clip

Dataset Validation Checklist

- ✓ No negative Frequency or Monetary
- ✓ Recency > 0 (customer made at least 1 purchase)
- ✓ No null values in R, F, M

- ✓ Mean Frequency > 1 (customers have repeat purchases, not one-time only)
- ✓ Outliers (top 1% Monetary) identified; decide: keep or clip

05 · ML ENGINE — KMEANS + SILHOUETTE + PERSONA GENERATION

Production clustering with validation and business narrative output.

KMeans Model Architecture

Component	Implementation	Purpose	Output
Feature Input	Scaled RFM (n, 3)	Standardized R, F, M	3-dimensional vectors
KMeans Estimator	<code>`KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=42)`</code>	Partitions n-k clusters	Cluster labels [0, state=42]
Silhouette Analyzer	<code>`silhouette_score(X, labels)`</code>	Global cohesion metric	Score ∈ [-1, +1]
Silhouette Samples	<code>`silhouette_samples(X, labels)`</code>	Per-point cohesion	Array of scores
Hyperparameter Tuner	Loop k ∈ [2, 10]; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats as template	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, M distributions	Detect customer base shift	Boolean flag + p-values

Feature Input	Scaled RFM (n, 3)	Standardized R, F, M	3-dimensional vectors
KMeans Estimator	<code>`KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=42)`</code>	Partitions n-k clusters	Cluster labels [0, state=42]
Silhouette Analyzer	<code>`silhouette_score(X, labels)`</code>	Global cohesion metric	Score ∈ [-1, +1]
Silhouette Samples	<code>`silhouette_samples(X, labels)`</code>	Per-point cohesion	Array of scores
Hyperparameter Tuner	Loop k ∈ [2, 10]; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats as template	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, M distributions	Detect customer base shift	Boolean flag + p-values

Feature Input	Scaled RFM (n, 3)	Standardized R, F, M	3-dimensional vectors
KMeans Estimator	<code>`KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=42)`</code>	Partitions n-k clusters	Cluster labels [0, state=42]
Silhouette Analyzer	<code>`silhouette_score(X, labels)`</code>	Global cohesion metric	Score ∈ [-1, +1]
Silhouette Samples	<code>`silhouette_samples(X, labels)`</code>	Per-point cohesion	Array of scores
Hyperparameter Tuner	Loop k ∈ [2, 10]; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats as template	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, M distributions	Detect customer base shift	Boolean flag + p-values

KMeans Estimator	<code>`KMeans(n_clusters=k, init='k-means++', max_iter=300, n_init=10, random_state=42)`</code>	Partitions n-k clusters	Cluster labels [0, state=42]
Silhouette Analyzer	<code>`silhouette_score(X, labels)`</code>	Global cohesion metric	Score ∈ [-1, +1]

Silhouette Samples	`silhouette_samples(X, labels)`	Per-point cohesion	Array of scores
Hyperparameter Tuner	Loop $k \in [2, 10]$; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, D	Distributional base shift	Boolean flag + p-values

Silhouette Analyzer	`silhouette_score(X, labels)`	Global cohesion metric	Score $\in [-1, +1]$
Silhouette Samples	`silhouette_samples(X, labels)`	Per-point cohesion	Array of scores
Hyperparameter Tuner	Loop $k \in [2, 10]$; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, D	Distributional base shift	Boolean flag + p-values

Silhouette Samples	`silhouette_samples(X, labels)`	Per-point cohesion	Array of scores
Hyperparameter Tuner	Loop $k \in [2, 10]$; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, D	Distributional base shift	Boolean flag + p-values

Hyperparameter Tuner	Loop $k \in [2, 10]$; pick max silhouette	Find optimal cluster count	Best k + corresponding model
Persona Engine	Custom class; centroid stats	Translate cluster to narrative	JSON with descriptions
Drift Detector	Weekly KS test on R, F, D	Distributional base shift	Boolean flag + p-values

KMeans Hyperparameter Search

Parameter	Search Range	Best Value (Typical)	Rationale
`n_clusters` (k)	2 to 10	4-5 (business domain)	Too few = lose insight; too many = noise
`init`	{'k-means++', 'random'}	'k-means++'	Better initialization → faster convergence
`max_iter`	100 to 1000	300	Enough for convergence; diminishing returns after
`n_init`	1 to 20	10	Multiple random inits reduce local optima risk
`random_state`	Fixed 42	42	Reproducibility

`n_clusters` (k)	2 to 10	4-5 (business domain)	Too few = lose insight; too many = noise
`init`	{'k-means++', 'random'}	'k-means++'	Better initialization → faster convergence
`max_iter`	100 to 1000	300	Enough for convergence; diminishing returns after
`n_init`	1 to 20	10	Multiple random inits reduce local optima risk
`random_state`	Fixed 42	42	Reproducibility

`n_clusters` (k)	2 to 10	4-5 (business domain)	Too few = lose insight; too many = noise
`init`	{'k-means++', 'random'}	'k-means++'	Better initialization → faster convergence

`max_iter`	100 to 1000	300	Enough for convergence; diminishing returns at
`n_init`	1 to 20	10	Multiple random inits reduce local optima risk
`random_state`	Fixed 42	42	Reproducibility

`init`	{'k-means++', 'random'}	'k-means++'	Better initialization → faster convergence
`max_iter`	100 to 1000	300	Enough for convergence; diminishing returns at
`n_init`	1 to 20	10	Multiple random inits reduce local optima risk
`random_state`	Fixed 42	42	Reproducibility

`max_iter`	100 to 1000	300	Enough for convergence; diminishing returns at
`n_init`	1 to 20	10	Multiple random inits reduce local optima risk
`random_state`	Fixed 42	42	Reproducibility

Search strategy:

```
from sklearn.metrics import silhouette_score
import pandas as pd

silhouette_scores = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, init='k-means++', max_iter=300,
                random_state=42, n_init=10)
    labels = km.fit_predict(rfm_scaled)
    score = silhouette_score(rfm_scaled, labels)
    silhouette_scores.append({'k': k, 'silhouette': score})
    print(f"k={k}: silhouette={score:.3f}")

best_k = max(silhouette_scores, key=lambda x: x['silhouette'])['k']
print(f"Optimal k: {best_k}")
```

```
from sklearn.metrics import silhouette_score
import pandas as pd

silhouette_scores = []
for k in range(2, 11):
    km = KMeans(n_clusters=k, init='k-means++', max_iter=300,
                random_state=42, n_init=10)
    labels = km.fit_predict(rfm_scaled)
    score = silhouette_score(rfm_scaled, labels)
    silhouette_scores.append({'k': k, 'silhouette': score})
    print(f"k={k}: silhouette={score:.3f}")

best_k = max(silhouette_scores, key=lambda x: x['silhouette'])['k']
```

```

print(f"Optimal k: {best_k}")

### Silhouette Scoring — What It Means for Business

Silhouette score  $\in [-1, +1]$ :
- **Score > 0.5:** Well-separated clusters; actionable segments.
- **Score 0.25–0.5:** Overlapping clusters; still interpretable but less distinct.
- **Score < 0.25:** Poor segmentation; consider different features or more data.

**Target:** Silhouette  $\geq 0.50$  for portfolio credibility.

### Persona Generation from Cluster Centroids

**Input:** Fitted KMeans model + scaled RFM data.

**Logic:**
1. Inverse-scale cluster centroids back to original R, F, M ranges.
2. Compute per-cluster statistics: mean, median, std, min, max for each dimension.
3. Rank each cluster on R, F, M: "Low"/"Medium"/"High" based on quantile position.
4. Fill template narrative.

**Example template (Jinja2):**

```

Silhouette Scoring — What It Means for Business

Silhouette score $\in [-1, +1]$:

- Score > 0.5:** Well-separated clusters; actionable segments.
- Score 0.25–0.5:** Overlapping clusters; still interpretable but less distinct.
- Score < 0.25:** Poor segmentation; consider different features or more data.

Target: Silhouette ≥ 0.50 for portfolio credibility.

Persona Generation from Cluster Centroids

Input: Fitted KMeans model + scaled RFM data.

Logic:

1. Inverse-scale cluster centroids back to original R, F, M ranges.
2. Compute per-cluster statistics: mean, median, std, min, max for each dimension.
3. Rank each cluster on R, F, M: "Low"/"Medium"/"High" based on quantile position.
4. Fill template narrative.

Example template (Jinja2):

```
{% for cluster in clusters %}
```

```
{% for cluster in clusters %}
Cluster {{ cluster.label }}: {{ cluster.persona_name }}
```

RFM Profile:

- Recency: {{ cluster.r_rank }} (avg {{ cluster.r_mean | round(1) }} days since purchase)
- Frequency: {{ cluster.f_rank }} (avg {{ cluster.f_mean | round(1) }} purchases/year)
- Monetary: {{ cluster.m_rank }} (avg £{{ cluster.m_mean | round(2) }} lifetime value)

Cluster Size: {{ cluster.count }} customers ({{ cluster.pct | round(1) }}% of base)

Recommended Action:

```
{% if cluster.f_rank == 'High' and cluster.m_rank == 'High' %}
• VIP Segment**: High-value frequent buyers. Tier for premium support + early access.
{% elif cluster.f_rank == 'Low' and cluster.r_rank == 'High' %}
• At-Risk Segment**: Haven't purchased recently. Launch re-engagement campaign.
{% endif %}
{% endfor %}

**Example output:**
```

Example output:

Cluster 0: Platinum Buyers

RFM Profile:

- Recency: Recent (avg 18 days since purchase)
- Frequency: High (avg 87 purchases/year)
- Monetary: High (avg £12,540 lifetime value)

Cluster Size: 247 customers (4.9% of base)

Recommended Action:

- VIP Segment: High-value frequent buyers. Tier for premium support + early access.

```
### Model Evaluation Metrics

| Metric | Formula / Method | Target | Interpretation |
|-----|-----|-----|-----|
| **Silhouette Score** | `mean(silhouette_samples)` |  $\geq 0.50$  | Global cluster cohesion |
| **Silhouette per Cluster** | `silhouette_samples` per label |  $> 0.3$  each | No consistently bad clusters |
| **Davies-Bouldin Index** | Mean ratio of within/between variance |  $< 1.50$  | Lower = tighter, more separated |
| **Calinski-Harabasz Index** | Ratio of between to within variance |  $> 100$  (heuristic) | Higher = denser, better |
| **Inertia (WCSS)** | Sum of squared distances to centroids | Decreasing as  $k \uparrow$  | Elbow plot; diminishing return |
| **Persona Interpretability** | Manual review: can you explain why cluster makes sense? | Binary: Yes/No | Rule-
```

Model Evaluation Metrics

Metric	Formula / Method	Target	Interpretation
Silhouette Score	<code>`mean(silhouette_samples)`</code>	> 0.50	Global cluster cohesion
Silhouette per Cluster	<code>`silhouette_samples` per cluster</code>	> 0.2 each	No consistently bad clusters
Davies-Bouldin Index	Mean ratio of within/between variance	< 1.50	Lower = tighter, more separated
Calinski-Harabasz Index	Ratio of between to within variance (heuristic)	> 100	Higher = denser, better-separated
Inertia (WCSS)	Sum of squared distances to centroids	Decreasing as $k \uparrow$	Elbow plot; diminishing returns after optimal k
Persona Interpretability	Manual review: can you explain why cluster makes sense?	Yes/No	Rule-based check

Silhouette Score	<code>`mean(silhouette_samples)`</code>	> 0.50	Global cluster cohesion
Silhouette per Cluster	<code>`silhouette_samples` per cluster</code>	> 0.2 each	No consistently bad clusters
Davies-Bouldin Index	Mean ratio of within/between variance	< 1.50	Lower = tighter, more separated
Calinski-Harabasz Index	Ratio of between to within variance (heuristic)	> 100	Higher = denser, better-separated
Inertia (WCSS)	Sum of squared distances to centroids	Decreasing as $k \uparrow$	Elbow plot; diminishing returns after optimal k
Persona Interpretability	Manual review: can you explain why cluster makes sense?	Yes/No	Rule-based check

Silhouette Score	<code>`mean(silhouette_samples)`</code>	> 0.50	Global cluster cohesion
Silhouette per Cluster	<code>`silhouette_samples` per cluster</code>	> 0.2 each	No consistently bad clusters
Davies-Bouldin Index	Mean ratio of within/between variance	< 1.50	Lower = tighter, more separated
Calinski-Harabasz Index	Ratio of between to within variance (heuristic)	> 100	Higher = denser, better-separated
Inertia (WCSS)	Sum of squared distances to centroids	Decreasing as $k \uparrow$	Elbow plot; diminishing returns after optimal k
Persona Interpretability	Manual review: can you explain why cluster makes sense?	Yes/No	Rule-based check

Silhouette per Cluster	<code>`silhouette_samples` per cluster</code>	> 0.2 each	No consistently bad clusters
Davies-Bouldin Index	Mean ratio of within/between variance	< 1.50	Lower = tighter, more separated
Calinski-Harabasz Index	Ratio of between to within variance (heuristic)	> 100	Higher = denser, better-separated
Inertia (WCSS)	Sum of squared distances to centroids	Decreasing as $k \uparrow$	Elbow plot; diminishing returns after optimal k
Persona Interpretability	Manual review: can you explain why cluster makes sense?	Yes/No	Rule-based check

Davies-Bouldin Index	Mean ratio of within/between variance	< 1.50	Lower = tighter, more separated
Calinski-Harabasz Index	Ratio of between to within variance (heuristic)	> 100	Higher = denser, better-separated
Inertia (WCSS)	Sum of squared distances to centroids	Decreasing as $k \uparrow$	Elbow plot; diminishing returns after optimal k
Persona Interpretability	Manual review: can you explain why cluster makes sense?	Yes/No	Rule-based check

Calinski-Harabasz Index	Ratio of between to within-cluster variance	100% (perfect)	Higher = denser, better-separated
Inertia (WCSS)	Sum of squared distances to centroids	Decreasing as k ↑	Elbow plot; diminishing returns after optimal k
Persona Interpretability	Manual review: can you explain why cluster makes sense?	Binary Yes/No	Rule-based check

06 · CLUSTER FRAMEWORK

Business-meaningful cluster tiers with example personas and recommended actions.

The 4-Persona Architecture (Example for Retail)

Assuming **k=4** from silhouette optimization:

Tier	**Name**	**Size**	**RFM Profile**	**Lifetime Value**	**Action**
■ Tier 1	**Platinum**	~5%	R: Recent (15d) F: High (£12k+) M: High (£10k+)	£12,000	VIP program, exclusive offers
■ Tier 2	**Gold**	~20%	R: Recent (35d) F: Medium (£5,500–40/yr) M: Medium (£2.5k)	£5,500	Targeted promotions, loyalty rewards
■ Tier 3	**Silver**	~45%	R: Moderate (90d) F: Low (£1,500/yr) M: Low (£500)	£1,500	Engagement nurture, cross-sell
■ Tier 4	**At-Risk**	~30%	R: Old (180+d) F: Low (<£500) M: Low (<£500)	<£500	Win-back campaigns, re-engagement
■ Tier 1	**Platinum**	~5%	R: Recent (15d) F: High (£12k+) M: High (£10k+)	£12,000	VIP program, exclusive offers
■ Tier 2	**Gold**	~20%	R: Recent (35d) F: Medium (£5,500–40/yr) M: Medium (£2.5k)	£5,500	Targeted promotions, loyalty rewards
■ Tier 3	**Silver**	~45%	R: Moderate (90d) F: Low (£1,500/yr) M: Low (£500)	£1,500	Engagement nurture, cross-sell
■ Tier 4	**At-Risk**	~30%	R: Old (180+d) F: Low (<£500) M: Low (<£500)	<£500	Win-back campaigns, re-engagement
■ Tier 1	**Platinum**	~5%	R: Recent (15d) F: High (£12k+) M: High (£10k+)	£12,000	VIP program, exclusive offers
■ Tier 2	**Gold**	~20%	R: Recent (35d) F: Medium (£5,500–40/yr) M: Medium (£2.5k)	£5,500	Targeted promotions, loyalty rewards
■ Tier 3	**Silver**	~45%	R: Moderate (90d) F: Low (£1,500/yr) M: Low (£500)	£1,500	Engagement nurture, cross-sell
■ Tier 4	**At-Risk**	~30%	R: Old (180+d) F: Low (<£500) M: Low (<£500)	<£500	Win-back campaigns, re-engagement
■ Tier 2	**Gold**	~20%	R: Recent (35d) F: Medium (£5,500–40/yr) M: Medium (£2.5k)	£5,500	Targeted promotions, loyalty rewards
■ Tier 3	**Silver**	~45%	R: Moderate (90d) F: Low (£1,500/yr) M: Low (£500)	£1,500	Engagement nurture, cross-sell
■ Tier 4	**At-Risk**	~30%	R: Old (180+d) F: Low (<£500) M: Low (<£500)	<£500	Win-back campaigns, re-engagement

Exact Cluster Decision Rules (Derived from Centroids)

After KMeans fitting:

CLUSTER 0 (Platinum):

```
IF recency_scaled < P25 AND frequency_scaled > P75 AND monetary_scaled > P75:
    ASSIGN persona "Platinum" + actions
```

```
CLUSTER 1 (Gold):
    IF recency_scaled < P50 AND frequency_scaled > P50 AND monetary_scaled > P25:
        ASSIGN persona "Gold" + actions
```

```
CLUSTER 2 (Silver):
    IF recency_scaled ≈ MEDIAN AND frequency_scaled ≈ MEDIAN AND monetary_scaled ≈ MEDIAN:
        ASSIGN persona "Silver" + actions
```

```
CLUSTER 3 (At-Risk):
    IF recency_scaled > P75 OR frequency_scaled < P25:
        ASSIGN persona "At-Risk" + actions
```

CLUSTER 0 (Platinum):

IF recency_scaled < P25 AND frequency_scaled > P75 AND monetary_scaled > P75:

ASSIGN persona "Platinum" + actions

CLUSTER 1 (Gold):

IF recency_scaled < P50 AND frequency_scaled > P50 AND monetary_scaled > P25:

ASSIGN persona "Gold" + actions

CLUSTER 2 (Silver):

IF recency_scaled ≈ MEDIAN AND frequency_scaled ≈ MEDIAN AND monetary_scaled ≈ MEDIAN:

ASSIGN persona "Silver" + actions

CLUSTER 3 (At-Risk):

IF recency_scaled > P75 OR frequency_scaled < P25:

ASSIGN persona "At-Risk" + actions

Example Scored Customers

```
| Customer ID | Recency (d) | Frequency | Monetary (£) | Scaled RFM | **Cluster** | **Persona** | **Recommended A
|-----|-----|-----|-----|-----|-----|-----|
| CUST-00152 | 12 | 94 | 13,240 | (-1.1, 1.8, 1.9) | 0 | **Platinum** | Enroll VIP; premium support |
| CUST-04521 | 42 | 32 | 3,850 | (-0.2, 0.6, 0.8) | 1 | **Gold** | Loyalty rewards offer |
| CUST-07834 | 95 | 8 | 1,240 | (0.4, -0.5, -0.3) | 2 | **Silver** | Engagement drip campaign |
| CUST-11205 | 280 | 2 | 180 | (2.1, -1.2, -1.5) | 3 | **At-Risk** | Win-back email; exit survey |
```

Example Scored Customers

	Recency (d)	Frequency	Monetary (£)	Scaled RFM	**Cluster**	**Persona**
	12	94	13,240	(-1.1, 1.8, 1.9)	0	**Platinum**

	42	32	3,850	(−0.2, 0.6, 0.8)	1	**Gold**
	95	8	1,240	(0.4, −0.5, −0.3)	2	**Silver**
	280	2	180	(2.1, −1.2, −1.5)	3	**At-Risk**
	12	94	13,240	(−1.1, 1.8, 1.9)	0	**Platinum**
	42	32	3,850	(−0.2, 0.6, 0.8)	1	**Gold**
	95	8	1,240	(0.4, −0.5, −0.3)	2	**Silver**
	280	2	180	(2.1, −1.2, −1.5)	3	**At-Risk**
	12	94	13,240	(−1.1, 1.8, 1.9)	0	**Platinum**
	42	32	3,850	(−0.2, 0.6, 0.8)	1	**Gold**
	95	8	1,240	(0.4, −0.5, −0.3)	2	**Silver**
	280	2	180	(2.1, −1.2, −1.5)	3	**At-Risk**
	42	32	3,850	(−0.2, 0.6, 0.8)	1	**Gold**
	95	8	1,240	(0.4, −0.5, −0.3)	2	**Silver**
	280	2	180	(2.1, −1.2, −1.5)	3	**At-Risk**

07 · PHASE-BY-PHASE BUILD PLAN

12-week production-ready delivery roadmap (shorter than LeadForge; simpler domain).

PHASE 1 — Data Infrastructure & RFM Engineering (Weeks 1–2)

Goal: Ingest transaction data, compute RFM, validate features, version dataset in DVC.

Detailed Tasks:

- Clone UCI Online Retail dataset (or generate synthetic); store in `/data/raw/`.
- Pandas data cleaning: remove returns, consolidate duplicates, set reference date.
- RFM calculation: groupby CustomerID → Recency, Frequency, Monetary.
- Feature validation: nulls=0, Frequency>0, Monetary>0, Recency>0.
- Great Expectations suite: 15 expectations (dtypes, nulls, value ranges, distribution checks).
- StandardScaler fit on training subset (first 80% of date range); save scaler pickle.
- DVC init: track `raw.parquet`, `rfm.parquet`, `rfm\_scaled.parquet`.
- GitHub Actions CI: ruff, mypy, pytest on Jupyter notebook cells.
- MLflow experiment: "RFM-Baseline" created; scaler artifact logged.

Phase Exit Gates:

- ✓ ``/data/rfm_scaled.parquet`` with `(n_customers, 3)` shape; all nulls removed.
- ✓ GE report: 15/15 expectations pass.
- ✓ DVC: ``dvc repro`` → reproducible RFM matrix.
- ✓ Scaler pickle saved to MLflow artefact store.

PHASE 2 — KMeans Model & Silhouette Optimization (Weeks 3–5)

Goal: Train, tune, and validate KMeans across $k \in [2, 10]$. Achieve Silhouette ≥ 0.50 . Generate personas.

Detailed Tasks:

- KMeans loop: fit $k=2$ to $k=10$; log `silhouette_score`, `inertia`, `centroids` to MLflow.
- Elbow plot: visualize `inertia` vs k ; `silhouette` score vs k .
- Select best k (highest silhouette); refit with `n_init=20` for robustness.
- Silhouette analysis: compute `silhouette_samples`; visualize per-cluster scores.
- Centroid inverse-scaling: convert scaled centroids back to R, F, M ranges.
- Persona generation: compute per-cluster stats (mean, median, std, quartiles); fill Jinja2 templates.
- Persona validation: manually review 5 sampled customers per cluster; confirm personas make sense.
- Cluster assignment: `predict(rfm_scaled)` → array of cluster labels.
- Davies-Bouldin + Calinski-Harabasz indices: log to MLflow as supplementary metrics.
- Model registry: save best KMeans model to MLflow as 'CustomerSegment-KMeans-v1'; stage 'Production'.
- Persona JSON export: cluster profiles, customer count per cluster, recommended actions.

Phase Exit Gates:

- ✓ Silhouette score ≥ 0.50 (or justified if lower due to data characteristics).
- ✓ Silhouette plot: all clusters show > 0.3 average (no outlier clusters).
- ✓ Persona descriptions: clear, actionable, non-overlapping.
- ✓ All 5k customers assigned to clusters; cluster distribution logged.

PHASE 3 — Streamlit Dashboard & Optional API (Weeks 6–10)

Goal: Build interactive Streamlit dashboard for cluster exploration. Optional: FastAPI scoring endpoint.

Streamlit Dashboard (5 tabs):

1. Cluster Overview:

- Pie chart: cluster size distribution.
- Table: cluster name, size, %age, silhouette score, mean R/F/M.
- Key metric: "Silhouette Score: 0.52" displayed prominently.

2. Cluster Profiles:

- Expandable cards per cluster.
- Persona narrative, RFM stats table, distribution plots (R, F, M histograms per cluster).
- "Recommended Action" section.

3. Member Explorer:

- Searchable table: all customers; columns: CustomerID, Cluster, R, F, M, LastPurchaseDate.
- Filter by cluster dropdown; sort by R/F/M.
- Click row → detail view: customer purchase history (if available).

4. Silhouette Analysis:

- Horizontal bar chart: silhouette score per customer (sorted by cluster).
- Threshold line at 0 (negative = potentially misclassified).
- Interpretation guide: "Scores > 0.3 = well-matched to cluster."

5. Segmentation Health:

- Davies-Bouldin Index (lower better); Calinski-Harabasz Index (higher better).
- Comparison to baseline (previous week's model if available).
- Drift detection: "RFM distribution shift: NO ALERT" or "ALERT: Frequency distribution changed ($p < 0.05$)."

Streamlit Code Outline:

```
import streamlit as st
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from sklearn.preprocessing import StandardScaler
import joblib

# Load data + model
rfm = pd.read_parquet('data/rfm.parquet')
rfm_scaled_array = joblib.load('models/rfm_scaled.pkl')
km_model = joblib.load('models/kmeans_best.pkl')
personas = pd.read_json('models/personas.json')

st.set_page_config(page_title="CustomerSegment AI", layout="wide")
```

```

st.title("■ CustomerSegment AI – Customer RFM Analytics")

# Tabs
tab1, tab2, tab3, tab4, tab5 = st.tabs([
    "Overview", "Profiles", "Members", "Silhouette", "Health"
])

with tab1:
    st.subheader("Cluster Distribution")
    # Pie chart of cluster sizes
    cluster_counts = pd.DataFrame({
        'cluster': rfm['cluster'],
    }).value_counts()
    fig = px.pie(cluster_counts, ...)
    st.plotly_chart(fig)

with tab2:
    st.subheader("Persona Profiles")
    for idx, persona in personas.iterrows():
        with st.expander(f"Cluster {idx}: {persona['name']}"):
            st.markdown(f"***Size:** {persona['count']} customers ({persona['pct']}%)")
            st.markdown(f"***Avg Recency:** {persona['r_mean']:.1f} days")
            # ... more stats

# etc.

import streamlit as st
import pandas as pd
import plotly.graph_objects as go
import plotly.express as px
from sklearn.preprocessing import StandardScaler
import joblib

```

Load data + model

```

rfm = pd.read_parquet('data/rfm.parquet')
rfm_scaled_array = joblib.load('models/rfm_scaled.pkl')
km_model = joblib.load('models/kmeans_best.pkl')
personas = pd.read_json('models/personas.json')

st.set_page_config(page_title="CustomerSegment AI", layout="wide")
st.title("■ CustomerSegment AI — Customer RFM Analytics")

```

Tabs

```
tab1, tab2, tab3, tab4, tab5 = st.tabs([
    "Overview", "Profiles", "Members", "Silhouette", "Health"
])
```

with tab1:

```
st.subheader("Cluster Distribution")
```

Pie chart of cluster sizes

```
cluster_counts = pd.DataFrame({
    'cluster': rfm['cluster'],
}).value_counts()
fig = px.pie(cluster_counts, ...)
st.plotly_chart(fig)
```

with tab2:

```
st.subheader("Persona Profiles")
```

```
for idx, persona in personas.iterrows():
```

```
    with st.expander(f"Cluster {idx}: {persona['name']}"):

```

```
        st.markdown(f"Size: {persona['count']} customers ({persona['pct']}%)")

```

```
        st.markdown(f"Avg Recency: {persona['r_mean']:.1f} days")
```

... more stats

etc.

****Optional FastAPI Endpoint (for real-time customer assignment):****

Optional FastAPI Endpoint (for real-time customer assignment):

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class RFMInput(BaseModel):
    recency: float
    frequency: float
    monetary: float
```

```

@app.post("/predict-cluster")
async def predict(rfm: RFMInput):
    # Scale using fitted scaler
    scaled = scaler.transform([[rfm.recency, rfm.frequency, rfm.monetary]])
    cluster = km_model.predict(scaled)[0]
    persona = personas.loc[cluster, 'name']
    return {
        "cluster": int(cluster),
        "persona": persona,
        "recommendation": personas.loc[cluster, 'action']
    }

```

```

from fastapi import FastAPI
from pydantic import BaseModel

```

```
app = FastAPI()
```

```
class RFMInput(BaseModel):
```

```
    recency: float
```

```
    frequency: float
```

```
    monetary: float
```

```
@app.post("/predict-cluster")
```

```
async def predict(rfm: RFMInput):
```

Scale using fitted scaler

```
scaled = scaler.transform([[rfm.recency, rfm.frequency, rfm.monetary]])
```

```
cluster = km_model.predict(scaled)[0]
```

```
persona = personas.loc[cluster, 'name']
```

```
return {
```

```
    "cluster": int(cluster),
```

```
    "persona": persona,
```

```
    "recommendation": personas.loc[cluster, 'action']
```

```
}
```

```

**Dashboard Deployment:**

```

- Streamlit Cloud (free): push to GitHub → auto-deploy.
- Hugging Face Spaces (free): upload app.py → auto-restart weekly.
- Railway.app (\$5/mo): custom domain + environment variables.

```

**Testing:**

```

- 10 Streamlit unit tests (mock data, tab rendering, chart generation).

- 5 FastAPI tests (mock scaler, cluster assignment correctness).

****Phase Exit Gates:****

- ✓ Streamlit app runs locally: `streamlit run app.py` → opens on <http://localhost:8501>.
- ✓ All 5 tabs render; no errors or missing data.
- ✓ Cluster Overview pie chart + table match expected sizes.
- ✓ Member Explorer searchable; filter by cluster works.
- ✓ Silhouette bar chart renders correctly.
- ✓ Optional: FastAPI `/predict-cluster` endpoint tested with sample inputs.

PHASE 4 – Production Hardening & Documentation (Weeks 11-12)

****Goal:**** Deploy dashboard, set up weekly retraining, create documentation.

****Detailed Tasks:****

- Containerize: Dockerfile for Streamlit app (Python 3.11-slim, dependencies).
- Railway deployment: Connect GitHub repo; deploy to Railway.app.
- Weekly retraining job:
 - APScheduler: trigger every Sunday 23:00 UTC.
 - Pull new transactions from Supabase; recompute RFM.
 - Refit KMeans if silhouette drops > 5% OR customer count changes > 10%.
 - Log run to MLflow; update production model if metrics improve.
 - Slack notification: "Retraining run complete. Silhouette: 0.52. Model promoted."
- Drift monitoring:
 - Weekly KS test on R, F, M distributions (current week vs baseline).
 - Email alert if p-value < 0.05; include distribution comparison plots.
- Documentation:
 - `README.md`: overview, setup, local dev (clone → `pip install -r requirements.txt` → `streamlit run app.py`)
 - `METHODOLOGY.md`: RFM calculation, KMeans theory, persona generation logic.
 - `DEPLOYMENT.md`: Railway setup, environment variables, scheduled jobs.
 - `API.md`: (if FastAPI included) OpenAPI schema, example requests/responses.
- Demo video:
 - 5-minute walkthrough: navigate tabs, filter members, interpret silhouette scores.
 - Share on YouTube or Loom; link in GitHub README.

****Phase Exit Gates:****

- ✓ Streamlit app live on Railway or HF Spaces URL.
- ✓ Weekly retraining scheduled; test run successful.
- ✓ Drift monitoring alert tested (simulated data shift).
- ✓ Documentation complete: README, METHODOLOGY, DEPLOYMENT.
- ✓ Demo video uploaded; link in README.

Dashboard Deployment:

- Streamlit Cloud (free): push to GitHub → auto-deploy.
- Hugging Face Spaces (free): upload app.py → auto-restart weekly.
- Railway.app (\$5/mo): custom domain + environment variables.

Testing:

- 10 Streamlit unit tests (mock data, tab rendering, chart generation).
- 5 FastAPI tests (mock scaler, cluster assignment correctness).

Phase Exit Gates:

- ✓ Streamlit app runs locally: ``streamlit run app.py`` → opens on `http://localhost:8501`.
- ✓ All 5 tabs render; no errors or missing data.
- ✓ Cluster Overview pie chart + table match expected sizes.
- ✓ Member Explorer searchable; filter by cluster works.
- ✓ Silhouette bar chart renders correctly.
- ✓ Optional: FastAPI ``/predict-cluster`` endpoint tested with sample inputs.

PHASE 4 — Production Hardening & Documentation (Weeks 11–12)

Goal: Deploy dashboard, set up weekly retraining, create documentation.

Detailed Tasks:

- Containerize: Dockerfile for Streamlit app (Python 3.11-slim, dependencies).
- Railway deployment: Connect GitHub repo; deploy to Railway.app.
- Weekly retraining job:
 - APScheduler: trigger every Sunday 23:00 UTC.
 - Pull new transactions from Supabase; recompute RFM.
 - Refit KMeans if silhouette drops > 5% OR customer count changes > 10%.
 - Log run to MLflow; update production model if metrics improve.
 - Slack notification: "Retraining run complete. Silhouette: 0.52. Model promoted."
- Drift monitoring:
 - Weekly KS test on R, F, M distributions (current week vs baseline).
 - Email alert if p-value < 0.05; include distribution comparison plots.
- Documentation:

- `README.md`: overview, setup, local dev (clone → `pip install -r requirements.txt` → `streamlit run app.py`).
- `METHODOLOGY.md`: RFM calculation, KMeans theory, persona generation logic.
- `DEPLOYMENT.md`: Railway setup, environment variables, scheduled jobs.
- `API.md`: (if FastAPI included) OpenAPI schema, example requests/responses.
- Demo video:
- 5-minute walkthrough: navigate tabs, filter members, interpret silhouette scores.
- Share on YouTube or Loom; link in GitHub README.

Phase Exit Gates:

- ✓ Streamlit app live on Railway or HF Spaces URL.
- ✓ Weekly retraining scheduled; test run successful.
- ✓ Drift monitoring alert tested (simulated data shift).
- ✓ Documentation complete: README, METHODOLOGY, DEPLOYMENT.
- ✓ Demo video uploaded; link in README.

08 · STREAMLIT DASHBOARD DESIGN

Python-native analytics interface — no HTML/CSS complexity.

Design Approach

- Color scheme:** Dark mode by default (Streamlit's `theme.base="dark"`). Accent color: blue (`#1E88E5`) for clusters.
- Information density:** High; 4–5 visualizations per tab. Sales/marketing users expect data-rich views.
- Interactivity:** Dropdowns for cluster filtering; multiselect for range; search bar in Members tab.
- Latency:** All data loaded at app startup (< 1 second for 5k customers on modern laptop).

Tab 1: Overview

Title: "Cluster Distribution"

[Metric Cards]

■ Total Customers: 5,247 ■ Num Clusters: 4 ■ Silhouette Score: 0.52 ■

[Pie Chart: Cluster Size Distribution]

Platinum (5.2%, 273 customers) — blue

Gold (19.8%, 1,040 customers) — teal

Silver (44.2%, 2,322 customers) — gray

At-Risk (30.8%, 1,612 customers) — red

[Table: Cluster Summary]

Cluster	Size	%	Silhouette	Avg Recency	Avg Frequency	Avg Monetary
Platinum	273	5.2	0.68	18 days	87	£12,540
...						

Title: "Cluster Distribution"

[Metric Cards]

■ ■ Total Customers: 5,247

■ Num Clusters: 4

■ Silhouette Score: 0.52

■ ■

[Pie Chart: Cluster Size Distribution]

Platinum (5.2%, 273 customers) — blue

Gold (19.8%, 1,040 customers) — teal

Silver (44.2%, 2,322 customers) — gray

At-Risk (30.8%, 1,612 customers) — red

[Table: Cluster Summary]

Cluster	Size	%	Silhouette	Avg Recency	Avg Frequency	Avg Monetary
Platinum	273	5.2	0.68	18 days	87	£12,540
...						

Tab 2: Profiles

Tab 2: Profiles

Title: "Cluster Profiles"

[Cluster 0: Platinum Buyers] (Expandable)

RFM Summary:

- Recency: 18 days (Recent)
- Frequency: 87 purchases/year (High)
- Monetary: £12,540 lifetime value (High)

Recommended Action:

- ✓ Enroll in VIP loyalty program
- ✓ Early access to new products
- ✓ Dedicated support

[Histogram: Recency distribution in Platinum] [Histogram: Frequency] [Histogram: Monetary]

[Cluster 1: Gold Buyers]

... similar layout

Title: "Cluster Profiles"

[Cluster 0: Platinum Buyers] (Expandable)

RFM Summary:

- Recency: 18 days (Recent)
- Frequency: 87 purchases/year (High)
- Monetary: £12,540 lifetime value (High)

Recommended Action:

- ✓ Enroll in VIP loyalty program
- ✓ Early access to new products
- ✓ Dedicated support

[Histogram: Recency distribution in Platinum] [Histogram: Frequency] [Histogram: Monetary]

[Cluster 1: Gold Buyers]

... similar layout

Tab 3: Member Explorer

Tab 3: Member Explorer

Title: "Customer Explorer"

[Filter Controls]

Cluster: [Dropdown: All / Platinum / Gold / Silver / At-Risk]

Search: [Text input: search by CustomerID]

Sort by: [Dropdown: Recency / Frequency / Monetary]

[Paginated Table: top 50 rows displayed]

Customer ID	Cluster	Recency (d)	Frequency	Monetary (£)	Last Purchase
CUST-00152	Platinum	12	94	13,240	2024-01-18
CUST-04521	Gold	42	32	3,850	2024-01-07
...					

[Row detail on click]

Opens sidebar: Full customer profile + purchase history

Title: "Customer Explorer"

[Filter Controls]

Cluster: [Dropdown: All / Platinum / Gold / Silver / At-Risk]

Search: [Text input: search by CustomerID]

Sort by: [Dropdown: Recency / Frequency / Monetary]

[Paginated Table: top 50 rows displayed]

Customer ID | Cluster | Recency (d) | Frequency | Monetary (£) | Last Purchase

CUST-00152 | Platinum | 12 | 94 | 13,240 | 2024-01-18

CUST-04521 | Gold | 42 | 32 | 3,850 | 2024-01-07

...

[Row detail on click]

Opens sidebar: Full customer profile + purchase history

Tab 4: Silhouette Analysis

Tab 4: Silhouette Analysis

Title: "Cluster Cohesion Analysis"

[Silhouette Bar Chart]

Y-axis: Customer (sorted by cluster, then by silhouette score)

X-axis: Silhouette score [-1 to +1]

Color by cluster

Vertical line at x=0 (neutral threshold)

Vertical line at x=0.3 (interpretability threshold)

[Metrics table]

Silhouette Score: 0.52 (Global)

Platinum: 0.68

Gold: 0.55

Silver: 0.41

At-Risk: 0.51

Title: "Cluster Cohesion Analysis"

[Silhouette Bar Chart]

Y-axis: Customer (sorted by cluster, then by silhouette score)

X-axis: Silhouette score [-1 to +1]

Color by cluster

Vertical line at x=0 (neutral threshold)

Vertical line at x=0.3 (interpretability threshold)

[Metrics table]

Silhouette Score: 0.52 (Global)

Platinum: 0.68

Gold: 0.55

Silver: 0.41

At-Risk: 0.51

Tab 5: Segmentation Health

Tab 5: Segmentation Health

Title: "Model Quality & Drift Monitoring"

[Quality Metrics]

■■ Davies-Bouldin Index: 1.22 (Lower is better) ■■

■■ Calinski-Harabasz Index: 156.3 (Higher is better) ■■

■■ Inertia: 2,847 ■■

[Drift Monitoring]

Recency distribution: ✓ No change detected (KS p-value = 0.18)

Frequency distribution: ✓ No change detected (KS p-value = 0.42)

Monetary distribution: ✓ No change detected (KS p-value = 0.67)

Last recompute: 2024-01-22 (7 days ago)

Next recompute: 2024-01-29 (scheduled)

Title: "Model Quality & Drift Monitoring"

[Quality Metrics]

■■ Davies-Bouldin Index: 1.22 (Lower is better) ■■

■■ Calinski-Harabasz Index: 156.3 (Higher is better) ■■

■■ Inertia: 2,847 ■■

[Drift Monitoring]

Recency distribution: ✓ No change detected (KS p-value = 0.18)

Frequency distribution: ✓ No change detected (KS p-value = 0.42)

Monetary distribution: ✓ No change detected (KS p-value = 0.67)

Last recompute: 2024-01-22 (7 days ago)

Next recompute: 2024-01-29 (scheduled)

— — —

09 · PROMPT ENGINEERING LIBRARY

5 copy-paste-ready RISEN prompts for Claude / GPT-4o.

PROMPT 01 — RFM Feature Engineering & Validation

You are a senior data engineer specializing in customer analytics and RFM modeling.

TASK: Build a production RFM feature engineering pipeline for e-commerce transaction data.

DATASET: UCI Online Retail (500k transactions, 5k customers) or equivalent.

Columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice,
CustomerID, Country

RFM CALCULATION (exact):

1. Filter: Remove Quantity < 0 (returns); remove null CustomerID.
2. Per-customer aggregation (groupby CustomerID):
 - Last Purchase Date: MAX(InvoiceDate)
 - Recency: (reference_date - Last Purchase Date).days
 - Frequency: COUNT(DISTINCT InvoiceNo)
 - Monetary: SUM(Quantity × UnitPrice)
3. Reference date: MAX(InvoiceDate) in dataset (e.g., 2011-12-09)
4. Filter output: Keep only customers with Frequency > 0 AND Monetary > 0

FEATURE SCALING:

- StandardScaler: (X - mean) / std for [Recency, Frequency, Monetary]
- Fit scaler on 80% of data (chronologically early); apply to 100%
- Output: rfm_scaled.parquet with shape (n_customers, 3)

VALIDATION (Great Expectations):

- rfm['Recency'] > 0 AND <= 365 (all purchases within 1 year)
- rfm['Frequency'] >= 1 AND <= 1000
- rfm['Monetary'] > 0 AND not null
- No negative values in any column
- Distribution checks: Recency, Frequency NOT normal (expected; e-commerce is Pareto)

OUTPUT: Python module with RFMCalculator class:

- fit(df, reference_date) → scaler artifact
 - transform(df) → rfm_scaled (n, 3) array
 - inverse_transform(rfm_scaled) → rfm (original scale)
- Include: docstrings, type hints, 15 pytest tests with mock transaction data.

You are a senior data engineer specializing in customer analytics and RFM modeling.

TASK: Build a production RFM feature engineering pipeline for e-commerce transaction data.

DATASET: UCI Online Retail (500k transactions, 5k customers) or equivalent.

Columns: InvoiceNo, StockCode, Description, Quantity, InvoiceDate, UnitPrice,
CustomerID, Country

RFM CALCULATION (exact):

1. Filter: Remove Quantity < 0 (returns); remove null CustomerID.
2. Per-customer aggregation (groupby CustomerID):
 - Last Purchase Date: MAX(InvoiceDate)
 - Recency: (reference_date - Last Purchase Date).days
 - Frequency: COUNT(DISTINCT InvoiceNo)

- Monetary: $\text{SUM}(\text{Quantity} \times \text{UnitPrice})$

3. Reference date: $\text{MAX}(\text{InvoiceDate})$ in dataset (e.g., 2011-12-09)

4. Filter output: Keep only customers with $\text{Frequency} > 0$ AND $\text{Monetary} > 0$

FEATURE SCALING:

- StandardScaler: $(X - \text{mean}) / \text{std}$ for [Recency, Frequency, Monetary]
- Fit scaler on 80% of data (chronologically early); apply to 100%
- Output: rfm_scaled.parquet with shape (n_customers, 3)

VALIDATION (Great Expectations):

- $\text{rfm}[\text{'Recency'}] > 0$ AND ≤ 365 (all purchases within 1 year)
- $\text{rfm}[\text{'Frequency'}] \geq 1$ AND ≤ 1000
- $\text{rfm}[\text{'Monetary'}] > 0$ AND not null
- No negative values in any column
- Distribution checks: Recency, Frequency NOT normal (expected; e-commerce is Pareto)

OUTPUT: Python module with RFMCalculator class:

- $\text{fit}(\text{df}, \text{reference_date}) \rightarrow$ scaler artifact
- $\text{transform}(\text{df}) \rightarrow \text{rfm_scaled}$ (n, 3) array
- $\text{inverse_transform}(\text{rfm_scaled}) \rightarrow \text{rfm}$ (original scale)

Include: docstrings, type hints, 15 pytest tests with mock transaction data.

```
### PROMPT 02 – KMeans Clustering with Silhouette Optimization
```

PROMPT 02 — KMeans Clustering with Silhouette Optimization

You are a senior ML engineer specializing in unsupervised learning and clustering validation.

TASK: Build a robust KMeans clustering pipeline with silhouette-based k optimization.

INPUT: RFM scaled array (n_customers, 3); no missing values.

KMEANS CONFIG:

- sklearn.cluster.KMeans
- Grid search: n_clusters k $\in [2, 3, 4, 5, 6, 7, 8, 9, 10]$
- For each k: $\text{KMeans}(\text{n_clusters}=k, \text{init}=\text{'k-means++'}, \text{max_iter}=300, \text{random_state}=42, \text{n_init}=10)$
- Fit and compute $\text{silhouette_score}(X, \text{labels})$

SELECTION CRITERIA:

- Primary: $\text{argmax}(\text{silhouette_score across } k)$
- Secondary: If silhouette plateau ($\text{diff} < 0.02$ for consecutive k), prefer lower k (parsimony)
- Minimum threshold: Silhouette ≥ 0.30 (else warn user; data may be overlapping)

VALIDATION METRICS (compute for best k):

- silhouette_score (global): [-1, +1]; target ≥ 0.50
- silhouette_samples: per-customer array; flag negatives (misclassified)
- davies_bouldin_score: lower is better; < 1.5 excellent
- calinski_harabasz_score: higher is better; > 100 good
- inertia: within-cluster sum of squares (WCSS); for elbow plot

CENTROID INVERSE-SCALING:

- km.cluster_centers_ are in scaled space; inverse-transform to original R, F, M
- Output: centroids_original shape (k, 3) with Recency, Frequency, Monetary in original units

OUTPUT: Python module with ClusteringEngine class:

- fit(rfm_scaled) → finds best k, fits KMeans, logs all metrics
 - predict(rfm_scaled) → cluster labels
 - get_centroids_original() → DataFrame with (k, 3) centroids in R, F, M space
 - get_silhouette_samples() → array for per-customer visualization
- Include: MLflow logging (silhouette, centroids, best_k), 20 pytest tests.

You are a senior ML engineer specializing in unsupervised learning and clustering validation.

TASK: Build a robust KMeans clustering pipeline with silhouette-based k optimization.

INPUT: RFM scaled array (n_customers, 3); no missing values.

KMEANS CONFIG:

- sklearn.cluster.KMeans
- Grid search: n_clusters k $\in [2, 3, 4, 5, 6, 7, 8, 9, 10]$
- For each k: KMeans(n_clusters=k, init='k-means++', max_iter=300, random_state=42, n_init=10)
- Fit and compute silhouette_score(X, labels)

SELECTION CRITERIA:

- Primary: $\text{argmax}(\text{silhouette_score across } k)$
- Secondary: If silhouette plateau (diff < 0.02 for consecutive k), prefer lower k (parsimony)
- Minimum threshold: Silhouette ≥ 0.30 (else warn user; data may be overlapping)

VALIDATION METRICS (compute for best k):

- silhouette_score (global): [-1, +1]; target ≥ 0.50
- silhouette_samples: per-customer array; flag negatives (misclassified)
- davies_bouldin_score: lower is better; < 1.5 excellent
- calinski_harabasz_score: higher is better; > 100 good
- inertia: within-cluster sum of squares (WCSS); for elbow plot

CENTROID INVERSE-SCALING:

- `km.cluster_centers_` are in scaled space; inverse-transform to original R, F, M
- Output: `centroids_original` shape (k, 3) with Recency, Frequency, Monetary in original units

OUTPUT: Python module with `ClusteringEngine` class:

- `fit(rfm_scaled)` → finds best k, fits KMeans, logs all metrics
- `predict(rfm_scaled)` → cluster labels
- `get_centroids_original()` → DataFrame with (k, 3) centroids in R, F, M space
- `get_silhouette_samples()` → array for per-customer visualization

Include: MLflow logging (silhouette, centroids, best_k), 20 pytest tests.

```
### PROMPT 03 – Persona Generation from Cluster Statistics
```

PROMPT 03 — Persona Generation from Cluster Statistics

You are a senior analyst and data storyteller; your job is to translate cluster statistics into business-readable customer personas.

TASK: Generate persona descriptions and business recommendations from KMeans cluster statistics.

INPUT:

- `rfm` DataFrame: (n_customers, 4) with columns [CustomerID, Recency, Frequency, Monetary, cluster]
- `km_model`: fitted KMeans with k clusters
- `centroids_original`: (k, 3) array of cluster centers in R, F, M original units
- `persona_template`: Jinja2 template string (provided below)

PERSONA GENERATION LOGIC:

- For each cluster i:
 - Count cluster members; compute % of total
 - Compute per-cluster statistics:
 - mean, median, std, Q1, Q3 for R, F, M
 - compare to global mean: `ratio_r = cluster_mean_r / global_mean_r` (etc.)
 - Rank cluster on each dimension:
 - Recency: "Recent" if rank < 0.33, "Moderate" if 0.33-0.67, "Old" if > 0.67
 - Frequency: "Low" if rank < 0.33, "Medium" if 0.33-0.67, "High" if > 0.67
 - Monetary: "Low" if rank < 0.33, "Medium" if 0.33-0.67, "High" if > 0.67
- Assign persona name:
 - Decision tree based on (frequency_rank, monetary_rank, recency_rank)
 - E.g., (High, High, Recent) → "Platinum"; (Low, Low, Old) → "At-Risk"
- Assign business action:
 - (High, High, *): VIP program, premium support, exclusive access
 - (Medium, Medium, Recent): Loyalty rewards, targeted upsells
 - (Low, Low, *) AND (*, *, Old): Win-back campaigns, exit surveys
 - etc.
- Fill Jinja2 template with computed values

TEMPLATE (Jinja2):

```

Cluster {{ cluster.id }}: {{ cluster.persona_name }}
=====
Size: {{ cluster.count }} customers ({{ cluster.pct | round(1) }}%)

RFM Profile:
- Recency: {{ cluster.r_rank }} (avg {{ cluster.r_mean | round(1) }} days, std {{ cluster.r_std | round(1) }})
- Frequency: {{ cluster.f_rank }} (avg {{ cluster.f_mean | round(1) }} purchases, std {{ cluster.f_std | round(1) }})
- Monetary: {{ cluster.m_rank }} (avg £{{ cluster.m_mean | round(2) }}, std £{{ cluster.m_std | round(2) }})

Key Insight: {{ cluster.insight }}

Recommended Actions:
{% for action in cluster.actions %}
- {{ action }}
{% endfor %}

OUTPUT: Python module with PersonaGenerator class:
- fit(rfm_df, km_model) → compute all statistics
- generate_personas() → list of dicts: {id, name, count, r_mean, f_mean, m_mean, actions, insight}
- to_json() → JSON file of all personas
- to_markdown() → markdown report for documentation
Include: 15 pytest tests with mock clustering results.

```

You are a senior analyst and data storyteller; your job is to translate cluster statistics into business-readable customer personas.

TASK: Generate persona descriptions and business recommendations from KMeans cluster statistics.

INPUT:

- rfm DataFrame: (n_customers, 4) with columns [CustomerID, Recency, Frequency, Monetary, cluster]
- km_model: fitted KMeans with k clusters
- centroids_original: (k, 3) array of cluster centers in R, F, M original units
- persona_template: Jinja2 template string (provided below)

PERSONA GENERATION LOGIC:

1. For each cluster i:
 - a. Count cluster members; compute % of total
 - b. Compute per-cluster statistics:
 - mean, median, std, Q1, Q3 for R, F, M
 - compare to global mean: $\text{ratio}_r = \text{cluster_mean}_r / \text{global_mean}_r$ (etc.)
 - c. Rank cluster on each dimension:
 - Recency: "Recent" if rank < 0.33, "Moderate" if 0.33–0.67, "Old" if > 0.67
 - Frequency: "Low" if rank < 0.33, "Medium" if 0.33–0.67, "High" if > 0.67
 - Monetary: "Low" if rank < 0.33, "Medium" if 0.33–0.67, "High" if > 0.67

2. Assign persona name:

- Decision tree based on (frequency_rank, monetary_rank, recency_rank)
- E.g., (High, High, Recent) → "Platinum"; (Low, Low, Old) → "At-Risk"

3. Assign business action:

- (High, High, *): VIP program, premium support, exclusive access
- (Medium, Medium, Recent): Loyalty rewards, targeted upsells
- (Low, Low, *) AND (*, *, Old): Win-back campaigns, exit surveys
- etc.

4. Fill Jinja2 template with computed values

TEMPLATE (Jinja2):

Cluster {{ cluster.id }}: {{ cluster.persona_name }}

=====

Size: {{ cluster.count }} customers ({{ cluster.pct | round(1) }}%)

RFM Profile:

- Recency: {{ cluster.r_rank }} (avg {{ cluster.r_mean | round(1) }} days, std {{ cluster.r_std | round(1) }}
- Frequency: {{ cluster.f_rank }} (avg {{ cluster.f_mean | round(1) }} purchases, std {{ cluster.f_std | round(1) }}
- Monetary: {{ cluster.m_rank }} (avg £{{ cluster.m_mean | round(2) }}, std £{{ cluster.m_std | round(2) }}

Key Insight: {{ cluster.insight }}

Recommended Actions:

{% for action in cluster.actions %}

- {{ action }}

{% endfor %}

OUTPUT: Python module with PersonaGenerator class:

- fit(rfm_df, km_model) → compute all statistics
- generate_personas() → list of dicts: {id, name, count, r_mean, f_mean, m_mean, actions, insight}
- to_json() → JSON file of all personas
- to_markdown() → markdown report for documentation

Include: 15 pytest tests with mock clustering results.

PROMPT 04 – Streamlit Dashboard for Cluster Exploration

PROMPT 04 — Streamlit Dashboard for Cluster Exploration

You are a senior Streamlit engineer building an interactive customer segmentation dashboard.

TASK: Build a 5-tab Streamlit app for RFM clustering exploration and member discovery.

TECH: Streamlit 1.x, Plotly 5.x, Pandas 2.x, scikit-learn 1.x

TABS & COMPONENTS:

Tab 1 — Overview:

- Title: "Cluster Distribution"
- [st.metric] 3 columns: Total Customers, Num Clusters, Silhouette Score
- [plotly.pie] Cluster size pie chart (color by cluster)
- [st.dataframe] Summary table: cluster, size, %, silhouette, mean R/F/M

Tab 2 — Profiles:

- [st.expander] per cluster (title: "Cluster {i}: {persona_name}")
 - Persona narrative (st.markdown)
 - RFM summary stats (st.metric × 3 or table)
 - [plotly.histogram] 3 cols: distribution of R, F, M in cluster
 - Recommended actions (bulleted list)

Tab 3 — Member Explorer:

- [st.selectbox] Filter by cluster ("All" / "Platinum" / ...)
- [st.text_input] Search by CustomerID (substring match)
- [st.selectbox] Sort by (Recency / Frequency / Monetary)
- [st.dataframe] Paginated table: top 100 rows; columns: CustomerID, Cluster, R, F, M, LastDate
 - Make rows clickable (st.expander per row) to show customer detail

Tab 4 — Silhouette Analysis:

- [st.subheader] "Cluster Cohesion"
- [plotly.bar] Horizontal bar chart: silhouette_samples(X, labels)
 - Y-axis: Customer ID (sorted by cluster, then by score)
 - X-axis: Silhouette score [-1 to +1]
 - Color by cluster
 - Add vertical lines: x=0 (neutral), x=0.3 (threshold)
- [st.dataframe] Metrics per cluster: silhouette_score, davies_bouldin, calinski_harabasz

Tab 5 — Segmentation Health:

- [st.metric] Quality indices: Davies-Bouldin, Calinski-Harabasz, Inertia
- Drift monitoring section:
 - [st.dataframe] Per-dimension drift test (KS test p-values)
 - Status: "✓ No Drift" or "■■■ ALERT: Frequency distribution shifted (p=0.03)"
- Last recompute timestamp; next scheduled recompute

CODE OUTLINE:

```
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

# Load data
rfm = pd.read_parquet('data/rfm.parquet')
```

```

km_model = joblib.load('models/kmeans.pkl')
personas = json.load(open('models/personas.json'))
silhouette_samples = np.load('models/silhouette_samples.npy')

st.set_page_config(page_title="CustomerSegment AI", layout="wide")
st.title("■ CustomerSegment AI")

tab1, tab2, tab3, tab4, tab5 = st.tabs(["Overview", "Profiles", "Members", "Silhouette", "Health"])

with tab1:
    # Pie chart + summary table
    ...

with tab2:
    # Expandable persona cards
    ...

# etc.

OUTPUT: Complete app.py, requirements.txt, Dockerfile, docker-compose.yml.
Include: 10 Streamlit unit tests (using streamlit.testing.v1).

```

You are a senior Streamlit engineer building an interactive customer segmentation dashboard.

TASK: Build a 5-tab Streamlit app for RFM clustering exploration and member discovery.

TECH: Streamlit 1.x, Plotly 5.x, Pandas 2.x, scikit-learn 1.x

TABS & COMPONENTS:

Tab 1 — Overview:

- Title: "Cluster Distribution"
- [st.metric] 3 columns: Total Customers, Num Clusters, Silhouette Score
- [plotly.pie] Cluster size pie chart (color by cluster)
- [st.dataframe] Summary table: cluster, size, %, silhouette, mean R/F/M

Tab 2 — Profiles:

- [st.expander] per cluster (title: "Cluster {i}: {persona_name}")
- Persona narrative (st.markdown)
- RFM summary stats (st.metric × 3 or table)
- [plotly.histogram] 3 cols: distribution of R, F, M in cluster
- Recommended actions (bulleted list)

Tab 3 — Member Explorer:

- [st.selectbox] Filter by cluster ("All" / "Platinum" / ...)
- [st.text_input] Search by CustomerID (substring match)

- [st.selectbox] Sort by (Recency / Frequency / Monetary)
- [st.dataframe] Paginated table: top 100 rows; columns: CustomerID, Cluster, R, F, M, LastDate
- Make rows clickable (st.expander per row) to show customer detail

Tab 4 — Silhouette Analysis:

- [st.subheader] "Cluster Cohesion"
- [plotly.bar] Horizontal bar chart: silhouette_samples(X, labels)
- Y-axis: Customer ID (sorted by cluster, then by score)
- X-axis: Silhouette score [-1 to +1]
- Color by cluster
- Add vertical lines: x=0 (neutral), x=0.3 (threshold)
- [st.dataframe] Metrics per cluster: silhouette_score, davies_bouldin, calinski_harabasz

Tab 5 — Segmentation Health:

- [st.metric] Quality indices: Davies-Bouldin, Calinski-Harabasz, Inertia
- Drift monitoring section:
- [st.dataframe] Per-dimension drift test (KS test p-values)
- Status: "✓ No Drift" or "■■■ ALERT: Frequency distribution shifted (p=0.03)"
- Last recompute timestamp; next scheduled recompute

CODE OUTLINE:

```
import streamlit as st
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
```

Load data

```
rfm = pd.read_parquet('data/rfm.parquet')
km_model = joblib.load('models/kmeans.pkl')
personas = json.load(open('models/personas.json'))
silhouette_samples = np.load('models/silhouette_samples.npy')

st.set_page_config(page_title="CustomerSegment AI", layout="wide")
st.title("■ CustomerSegment AI")
```

```
tab1, tab2, tab3, tab4, tab5 = st.tabs(["Overview", "Profiles", "Members", "Silhouette", "Health"])
```

with tab1:

Pie chart + summary table

...

with tab2:

Expandable persona cards

...

etc.

OUTPUT: Complete app.py, requirements.txt, Dockerfile, docker-compose.yml.

Include: 10 Streamlit unit tests (using streamlit.testing.v1).

```
### PROMPT 05 — Weekly RFM Recomputation & Drift Monitoring
```

PROMPT 05 — Weekly RFM Recomputation & Drift Monitoring

You are a senior MLOps engineer building automated retraining and monitoring for a clustering pipeline.

TASK: Implement weekly RFM recalculation, KMeans refitting, and drift detection.

TECH: APScheduler 3.x, SQLAlchemy (optional; Supabase/SQLite), Pandas, scikit-learn, Logfire or Python logging, Slack webhook (notifications)

COMPONENTS:

1. WEEKLY SCHEDULER (APScheduler):
 - Trigger: Every Sunday 23:00 UTC
 - Job: run_weekly_segmentation()
 - Logging: Start time, end time, metrics, status (success/failure)
2. RFM RECOMPUTATION:
 - Query Supabase / SQLite: all transactions from last 365 days
 - Recompute Recency, Frequency, Monetary per customer
 - Compare to previous week's RFM stats (mean, std, distribution)
 - Run KS test (scipy.stats.ks_2samp) for each dimension
 - If any p-value < 0.05: flag as "drift detected"; log p-values
3. CONDITIONAL KMEANS REFITTING:
 - Check 1: Silhouette score drop from previous model > 5%? → Refit

- Check 2: Customer count change > 10%? → Refit
- Check 3: Drift detected in RFM distributions? → Refit
- If ANY check true: fit new KMeans; compute silhouette; compare to previous best
- If new silhouette $\geq$ (previous - 0.01): promote to production
- Else: keep previous model; log warning

4. ARTIFACT UPDATES:

- Write new RFM to rfm.parquet (DVC track)
- Write new KMeans model to MLflow registry
- Write new cluster assignments to Supabase customer_clusters table
- Update Redis cache: HSET customer:{id} → {r, f, m, cluster, persona} (TTL 7d)

5. NOTIFICATIONS:

- Success: Slack message to #ml-ops channel with metrics
- Failure: Email alert to data_team@company.com with error traceback
- Drift detected: Slack message with KS test results + distribution comparison plots

PSEUDOCODE:

```
@scheduler.scheduled_job('cron', day_of_week='sun', hour=23)
def run_weekly_segmentation():
    try:
        logger.info("Starting weekly RFM recomputation")

        # 1. Fetch & compute RFM
        transactions = supabase.get_transactions(days=365)
        rfm_new = compute_rfm(transactions)

        # 2. Drift detection
        rfm_old = pd.read_parquet('data/rfm_previous.parquet')
        drift_results = {
            'recency_pval': ks_2samp(rfm_new['Recency'], rfm_old['Recency']).pvalue,
            'frequency_pval': ks_2samp(rfm_new['Frequency'], rfm_old['Frequency']).pvalue,
            'monetary_pval': ks_2samp(rfm_new['Monetary'], rfm_old['Monetary']).pvalue,
        }
        drift_detected = any(p < 0.05 for p in drift_results.values())

        # 3. Conditional KMeans refitting
        rfm_scaled = scaler.transform(rfm_new)
        if drift_detected or should_refit():
            km_new = KMeans(n_clusters=best_k, ...).fit(rfm_scaled)
            sil_new = silhouette_score(rfm_scaled, km_new.labels_)

            if sil_new >= sil_previous - 0.01:
                # Promote to production
                mlflow.register_model(...)
                update_supabase_assignments(km_new.labels_)
                logger.info(f"Model promoted. Silhouette: {sil_new:.3f}")
            else:
                logger.warning(f"New model worse ({sil_new:.3f} < {sil_previous:.3f}). Keeping previous.")

        # 4. Save artifacts + notify
        rfm_new.to_parquet('data/rfm.parquet')
        send_slack_notification({
            'status': 'SUCCESS',
            'silhouette': sil_previous,
```

```

        'drift_detected': drift_detected,
        'customers': len(rfm_new),
    })
    logger.info("Weekly segmentation complete")
except Exception as e:
    logger.error(f"Segmentation failed: {e}")
    send_email_alert(f"CustomerSegment retraining failed: {e}")

```

OUTPUT: Python module with SchedulerManager class:

- start() → initialize APScheduler
- run_weekly_segmentation() → orchestrate all steps

Include: logging setup, Slack/email integrations, 10 pytest tests (mocked Supabase).

You are a senior MLOps engineer building automated retraining and monitoring for a clustering pipeline.

TASK: Implement weekly RFM recalculation, KMeans refitting, and drift detection.

TECH: APScheduler 3.x, SQLAlchemy (optional; Supabase/SQLite), Pandas, scikit-learn, Logfire or Python logging, Slack webhook (notifications)

COMPONENTS:

1. WEEKLY SCHEDULER (APScheduler):

- Trigger: Every Sunday 23:00 UTC
- Job: run_weekly_segmentation()
- Logging: Start time, end time, metrics, status (success/failure)

2. RFM RECOMPUTATION:

- Query Supabase / SQLite: all transactions from last 365 days
- Recompute Recency, Frequency, Monetary per customer
- Compare to previous week's RFM stats (mean, std, distribution)
- Run KS test (scipy.stats.ks_2samp) for each dimension
- If any p-value < 0.05: flag as "drift detected"; log p-values

3. CONDITIONAL KMEANS REFITTING:

- Check 1: Silhouette score drop from previous model > 5%? → Refit
- Check 2: Customer count change > 10%? → Refit
- Check 3: Drift detected in RFM distributions? → Refit
- If ANY check true: fit new KMeans; compute silhouette; compare to previous best
- If new silhouette ≥ (previous - 0.01): promote to production
- Else: keep previous model; log warning

4. ARTIFACT UPDATES:

- Write new RFM to rfm.parquet (DVC track)
- Write new KMeans model to MLflow registry
- Write new cluster assignments to Supabase customer_clusters table
- Update Redis cache: HSET customer:{id} → {r, f, m, cluster, persona} (TTL 7d)

5. NOTIFICATIONS:

- Success: Slack message to #ml-ops channel with metrics
- Failure: Email alert to data_team@company.com with error traceback
- Drift detected: Slack message with KS test results + distribution comparison plots

PSEUDOCODE:

```
@scheduler.scheduled_job('cron', day_of_week='sun', hour=23)
def run_weekly_segmentation():
    try:
        logger.info("Starting weekly RFM recomputation")
```

1. Fetch & compute RFM

```
transactions = supabase.get_transactions(days=365)
rfm_new = compute_rfm(transactions)
```

2. Drift detection

```
rfm_old = pd.read_parquet('data/rfm_previous.parquet')
drift_results = {
    'recency_pval': ks_2samp(rfm_new['Recency'], rfm_old['Recency']).pvalue,
    'frequency_pval': ks_2samp(rfm_new['Frequency'], rfm_old['Frequency']).pvalue,
    'monetary_pval': ks_2samp(rfm_new['Monetary'], rfm_old['Monetary']).pvalue,
}
drift_detected = any(p < 0.05 for p in drift_results.values())
```

3. Conditional KMeans refitting

```
rfm_scaled = scaler.transform(rfm_new)
```

```
if drift_detected or should_refit():
    km_new = KMeans(n_clusters=best_k, ...).fit(rfm_scaled)
    sil_new = silhouette_score(rfm_scaled, km_new.labels_)
```

```
if sil_new >= sil_previous - 0.01:
```

Promote to production

```
mlflow.register_model(...)
update_supabase_assignments(km_new.labels_)
logger.info(f"Model promoted. Silhouette: {sil_new:.3f}")
else:
    logger.warning(f"New model worse ({sil_new:.3f} < {sil_previous:.3f}). Keeping previous.")
```

4. Save artifacts + notify

```
rfm_new.to_parquet('data/rfm.parquet')
send_slack_notification({
    'status': 'SUCCESS',
    'silhouette': sil_previous,
    'drift_detected': drift_detected,
    'customers': len(rfm_new),
})
logger.info("Weekly segmentation complete")
except Exception as e:
    logger.error(f"Segmentation failed: {e}")
    send_email_alert(f"CustomerSegment retraining failed: {e}")
```

OUTPUT: Python module with SchedulerManager class:

- start() → initialize APScheduler
- run_weekly_segmentation() → orchestrate all steps

Include: logging setup, Slack/email integrations, 10 pytest tests (mocked Supabase).

ML metrics for clustering validation; business metrics for go/no-go decisions.

ML Clustering Metrics

Metric	Formula / Method	Target	Interpretation
Silhouette Score	<code>`silhouette_score(X, labels)</code>	0.50 [-1, +1]	Global cluster cohesion; -1 = all wrong; +1 = perfect
Silhouette per Cluster	<code>`silhouette_samples(X, labels)</code>	0.13 per label	No consistently misclassified clusters
Davies-Bouldin Index	Mean(max ratio of within-cluster variance per cluster)	0.150	Lower = more compact, better-separated clusters
Calinski-Harabasz Index	Ratio of between-cluster to within-cluster variance	1.00	Higher = better-defined clusters
Inertia (WCSS)	Sum of squared distances to cluster centroids	Decreasing as k ↑	Elbow plot; diminishing returns
Dunn Index	Ratio of minimum inter-cluster distance to maximum intra-cluster distance	0.5 (distance)	Higher = more separated

Silhouette Score	<code>`silhouette_score(X, labels)</code>	0.50 [-1, +1]	Global cluster cohesion; -1 = all wrong; +1 = perfect
Silhouette per Cluster	<code>`silhouette_samples(X, labels)</code>	0.13 per label	No consistently misclassified clusters
Davies-Bouldin Index	Mean(max ratio of within-cluster variance per cluster)	0.150	Lower = more compact, better-separated clusters
Calinski-Harabasz Index	Ratio of between-cluster to within-cluster variance	1.00	Higher = better-defined clusters
Inertia (WCSS)	Sum of squared distances to cluster centroids	Decreasing as k ↑	Elbow plot; diminishing returns
Dunn Index	Ratio of minimum inter-cluster distance to maximum intra-cluster distance	0.5 (distance)	Higher = more separated

Silhouette Score	<code>`silhouette_score(X, labels)</code>	0.50 [-1, +1]	Global cluster cohesion; -1 = all wrong; +1 = perfect
Silhouette per Cluster	<code>`silhouette_samples(X, labels)</code>	0.13 per label	No consistently misclassified clusters
Davies-Bouldin Index	Mean(max ratio of within-cluster variance per cluster)	0.150	Lower = more compact, better-separated clusters
Calinski-Harabasz Index	Ratio of between-cluster to within-cluster variance	1.00	Higher = better-defined clusters
Inertia (WCSS)	Sum of squared distances to cluster centroids	Decreasing as k ↑	Elbow plot; diminishing returns
Dunn Index	Ratio of minimum inter-cluster distance to maximum intra-cluster distance	0.5 (distance)	Higher = more separated

Silhouette per Cluster	<code>`silhouette_samples(X, labels)</code>	0.13 per label	No consistently misclassified clusters
Davies-Bouldin Index	Mean(max ratio of within-cluster variance per cluster)	0.150	Lower = more compact, better-separated clusters
Calinski-Harabasz Index	Ratio of between-cluster to within-cluster variance	1.00	Higher = better-defined clusters
Inertia (WCSS)	Sum of squared distances to cluster centroids	Decreasing as k ↑	Elbow plot; diminishing returns
Dunn Index	Ratio of minimum inter-cluster distance to maximum intra-cluster distance	0.5 (distance)	Higher = more separated

Davies-Bouldin Index	Mean(max ratio of within-cluster variance per cluster)	0.150	Lower = more compact, better-separated clusters
Calinski-Harabasz Index	Ratio of between-cluster to within-cluster variance	1.00	Higher = better-defined clusters
Inertia (WCSS)	Sum of squared distances to cluster centroids	Decreasing as k ↑	Elbow plot; diminishing returns
Dunn Index	Ratio of minimum inter-cluster distance to maximum intra-cluster distance	0.5 (distance)	Higher = more separated

Calinski-Harabasz Index	Ratio of between-cluster to within-cluster variance	Higher is better	Higher = better-defined clusters
Inertia (WCSS)	Sum of squared distances of points to cluster centroids	Decreasing is k ↑	Elbow plot; diminishing returns
Dunn Index	Ratio of minimum inter-cluster distance to maximum intra-cluster distance	Higher is better	Higher = clusters separated

Business Impact Metrics (Post-Deployment)

Metric	Measurement	Baseline	Target
Platinum LTV	Actual lifetime value of customers in Platinum tier (90-day post-assignment)	£8,000	£12,000+
Platinum Conv.	Conversion rate (% who purchase in next 30 days)	25%	45%+
At-Risk Retention	% of At-Risk customers retained after re-engagement campaign	10%	20%+
Segmentation Actionability	Survey: % marketers who say personas are "very actionable"	60%	80%+
Campaign ROI Lift	ROI of segment-targeted campaigns vs. untargeted	1.0x	2.5x+
Dashboard Adoption	% of marketing team using dashboard weekly	50%	70%+

Platinum LTV	Actual lifetime value of customers in Platinum tier (90-day post-assignment)	£8,000	£12,000+
Platinum Conv.	Conversion rate (% who purchase in next 30 days)	25%	45%+
At-Risk Retention	% of At-Risk customers retained after re-engagement campaign	10%	20%+
Segmentation Actionability	Survey: % marketers who say personas are "very actionable"	60%	80%+
Campaign ROI Lift	ROI of segment-targeted campaigns vs. untargeted	1.0x	2.5x+
Dashboard Adoption	% of marketing team using dashboard weekly	50%	70%+

Platinum LTV	Actual lifetime value of customers in Platinum tier (90-day post-assignment)	£8,000	£12,000+
Platinum Conv.	Conversion rate (% who purchase in next 30 days)	25%	45%+
At-Risk Retention	% of At-Risk customers retained after re-engagement campaign	10%	20%+
Segmentation Actionability	Survey: % marketers who say personas are "very actionable"	60%	80%+
Campaign ROI Lift	ROI of segment-targeted campaigns vs. untargeted	1.0x	2.5x+
Dashboard Adoption	% of marketing team using dashboard weekly	50%	70%+

Platinum Conv.	Conversion rate (% who purchase in next 30 days)	25%	45%+
At-Risk Retention	% of At-Risk customers retained after re-engagement campaign	10%	20%+
Segmentation Actionability	Survey: % marketers who say personas are "very actionable"	60%	80%+
Campaign ROI Lift	ROI of segment-targeted campaigns vs. untargeted	1.0x	2.5x+
Dashboard Adoption	% of marketing team using dashboard weekly	50%	70%+

At-Risk Retention	% of At-Risk customers retained after re-engagement campaign	10%	20%+
Segmentation Actionability	Survey: % marketers who say personas are "very actionable"	60%	80%+

Campaign ROI Lift	ROI of segment-targeted campaigns vs. untargeted	1.2x	2.5x+
Dashboard Adoption	% of marketing team using dashboard weekly		70%+

Segmentation Actionability	Survey: % marketers who say personas are	80%	very actionable*
Campaign ROI Lift	ROI of segment-targeted campaigns vs. untargeted	1.2x	2.5x+
Dashboard Adoption	% of marketing team using dashboard weekly		70%+

11 · RISK REGISTER & DEMO STRATEGY

Technical risks and how to win stakeholder demos.

Technical Risks

Risk	Likelihood	Impact	Mitigation
Poor silhouette (<0.3); overlapping clusters	MEDIUM	MEDIUM	Try alternative features (log-transform F/M); use
Sparse/missing customer IDs in transactions	HIGH	HIGH	Dedup logic; test on real dataset early (Phase 1
KMeans instability (different k across runs)	LOW	MEDIUM	Use `random_state=42`; fit multiple seeds; log
Feature scaling errors (unscaled data to KMeans)	MEDIUM	HIGH	Use sklearn Pipeline; unit tests on scaler fit/tran
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refine decis
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;
Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days

Poor silhouette (<0.3); overlapping clusters	MEDIUM	MEDIUM	Try alternative features (log-transform F/M); use
Sparse/missing customer IDs in transactions	HIGH	HIGH	Dedup logic; test on real dataset early (Phase 1
KMeans instability (different k across runs)	LOW	MEDIUM	Use `random_state=42`; fit multiple seeds; log
Feature scaling errors (unscaled data to KMeans)	MEDIUM	HIGH	Use sklearn Pipeline; unit tests on scaler fit/tran
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refine decis
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;
Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days

Poor silhouette (<0.3); overlapping clusters	MEDIUM	MEDIUM	Try alternative features (log-transform F/M); use
Sparse/missing customer IDs in transactions	HIGH	HIGH	Dedup logic; test on real dataset early (Phase 1
KMeans instability (different k across runs)	LOW	MEDIUM	Use `random_state=42`; fit multiple seeds; log
Feature scaling errors (unscaled data to KMeans)	MEDIUM	HIGH	Use sklearn Pipeline; unit tests on scaler fit/tran
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refine decis
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;

Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days"
Sparse/missing customer IDs in transactions	HIGH	HIGH	Dedup logic; test on real dataset early
KMeans instability (different k across runs)	LOW	MEDIUM	Use `random_state=42`; fit multiple seeds; log
Feature scaling errors (unscaled data to KMeans)	MEDIUM	HIGH	Use sklearn Pipeline; unit tests on scaler fit/trans
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refine decis
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;
Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days"
KMeans instability (different k across runs)	LOW	MEDIUM	Use `random_state=42`; fit multiple see
Feature scaling errors (unscaled data to KMeans)	MEDIUM	HIGH	Use sklearn Pipeline; unit tests on scaler fit/trans
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refine decis
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;
Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days"
Feature scaling errors (unscaled data to KMeans)	MEDIUM	HIGH	Use sklearn Pipeline; unit tests on scal
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refine decis
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;
Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days"
Persona descriptions too generic or contradictory	MEDIUM	MEDIUM	Manual QA of generated personas; refin
Weekly retraining fails silently	MEDIUM	MEDIUM	Comprehensive logging; Slack alerts on failure;
Customer churn (old IDs drop out); cluster stability	MEDIUM	LOW	Document: "cluster assignments valid 365 days"

How to Win the Stakeholder Demo in 5 Minutes

- Minute 1:** Show the Cluster Overview pie chart. Say: "We've divided your 5k customers into 4 personas. Platinum is 5% of your base but likely 40% of revenue."
- Minute 2:** Click into Cluster 0 (Platinum). Show the persona card: "Average customer buys 87 times/year, £12.5k lifetime value, purchased 18 days ago. These are your VIPs."
- Minute 3:** Go to Member Explorer. Filter by Platinum. Show 10 rows. Say: "Now your team can instantly see who's in each segment, sort by recent, and send targeted campaigns."
- Minute 4:** Show the silhouette plot. Say: "This measures how well-defined our clusters are. Score of 0.52 means these are distinct, actionable segments — not arbitrary."
- Minute 5:** Show the health tab. Say: "We check every week if customer behavior is drifting. If it shifts, we recompute automatically. Your segmentation stays fresh."

12 · TIMELINE, MILESTONES & PRICING

12-week delivery plan and freelance pricing.

12-Week Milestone Roadmap

Week	Phase	Milestone	Exit Gate
1	Phase 1	Data ingestion + RFM calculation	Initial dataset with 5k+ customers; null checks passed
2	Phase 1	Feature scaling + DVC versioning	Model reproducible; GE validation 15/15 passed
3	Phase 2	KMeans grid search (k=2-10)	Silhouette curve plotted; best k identified
4	Phase 2	Silhouette validation + first model	$Silhouette \geq 0.50$; centroids logged to MLflow
5	Phase 2	Persona generation + interpretation	Persona descriptions written + manually validated
6	Phase 3	Streamlit Tab 1–2 (Overview)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Membership)	Segmentation logic working; silhouette bar chart rendered
8	Phase 3	Streamlit Tab 5 (Health Drift)	Drift detection logic working; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job scheduled
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT.md
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

1	Phase 1	Data ingestion + RFM calculation	Initial dataset with 5k+ customers; null checks passed
2	Phase 1	Feature scaling + DVC versioning	Model reproducible; GE validation 15/15 passed
3	Phase 2	KMeans grid search (k=2-10)	Silhouette curve plotted; best k identified
4	Phase 2	Silhouette validation + first model	$Silhouette \geq 0.50$; centroids logged to MLflow
5	Phase 2	Persona generation + interpretation	Persona descriptions written + manually validated
6	Phase 3	Streamlit Tab 1–2 (Overview)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Membership)	Segmentation logic working; silhouette bar chart rendered
8	Phase 3	Streamlit Tab 5 (Health Drift)	Drift detection logic working; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job scheduled
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT.md
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

1	Phase 1	Data ingestion + RFM calculation	Initial dataset with 5k+ customers; null checks passed
2	Phase 1	Feature scaling + DVC versioning	Model reproducible; GE validation 15/15 passed
3	Phase 2	KMeans grid search (k=2-10)	Silhouette curve plotted; best k identified

4	Phase 2	Silhouette validation + first model ≥ 0.50 ; centroids logged to MLflow	Silhouette ≥ 0.50 ; centroids logged to MLflow
5	Phase 2	Persona generation + interpretation descriptions written + manually validated	Interpretation descriptions written + manually validated
6	Phase 3	Streamlit Tab 1–2 (Overview of Profiles)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Membership & Silhouette)	Membership & Silhouette rendering; silhouette bar chart rendering
8	Phase 3	Streamlit Tab 5 (Health) + optional FastAPI	Optional FastAPI working; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed & live on Railway URL; retraining job scheduled
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT.md
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

2	Phase 1	Feature scaling + DVC versioning	Overcoming reproducibility; GE validation 1
3	Phase 2	KMeans grid search ($k=2-10$)	Silhouette curve plotted; best k identified
4	Phase 2	Silhouette validation + first model ≥ 0.50 ; centroids logged to MLflow	Silhouette ≥ 0.50 ; centroids logged to MLflow
5	Phase 2	Persona generation + interpretation descriptions written + manually validated	Interpretation descriptions written + manually validated
6	Phase 3	Streamlit Tab 1–2 (Overview of Profiles)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Membership & Silhouette)	Membership & Silhouette rendering; silhouette bar chart rendering
8	Phase 3	Streamlit Tab 5 (Health) + optional FastAPI	Optional FastAPI working; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed & live on Railway URL; retraining job scheduled
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT.md
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

3	Phase 2	KMeans grid search ($k=2-10$)	Silhouette curve plotted; best k identified
4	Phase 2	Silhouette validation + first model ≥ 0.50 ; centroids logged to MLflow	Silhouette ≥ 0.50 ; centroids logged to MLflow
5	Phase 2	Persona generation + interpretation descriptions written + manually validated	Interpretation descriptions written + manually validated
6	Phase 3	Streamlit Tab 1–2 (Overview of Profiles)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Membership & Silhouette)	Membership & Silhouette rendering; silhouette bar chart rendering
8	Phase 3	Streamlit Tab 5 (Health) + optional FastAPI	Optional FastAPI working; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed & live on Railway URL; retraining job scheduled
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT.md
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

4	Phase 2	Silhouette validation + first model ≥ 0.50 ; centroids logged to MLflow	Silhouette ≥ 0.50 ; centroids logged to MLflow
5	Phase 2	Persona generation + interpretation descriptions written + manually validated	Interpretation descriptions written + manually validated
6	Phase 3	Streamlit Tab 1–2 (Overview of Profiles)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Membership & Silhouette)	Membership & Silhouette rendering; silhouette bar chart rendering

8	Phase 3	Streamlit Tab 5 (Health) Dashboard	Optional FastAPI; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job s
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMEN
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

5	Phase 2	Persona generation	4 interpretat
6	Phase 3	Streamlit Tab 1–2 (Overview + Profiles)	Persona cards render correctly
7	Phase 3	Streamlit Tab 3–4 (Members + Silhouettes)	Search/silhouette rendering; silhouette bar chart rende
8	Phase 3	Streamlit Tab 5 (Health) Dashboard	Optional FastAPI; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job s
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMEN
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

6	Phase 3	Streamlit Tab 1–2 (Overview + Profiles)	Persona cards render corre
7	Phase 3	Streamlit Tab 3–4 (Members + Silhouettes)	Search/silhouette rendering; silhouette bar chart rende
8	Phase 3	Streamlit Tab 5 (Health) Dashboard	Optional FastAPI; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job s
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMEN
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

7	Phase 3	Streamlit Tab 3–4 (Members + Silhouettes)	Search/silhouette rendering; silhouette bar ch
8	Phase 3	Streamlit Tab 5 (Health) Dashboard	Optional FastAPI; /predict-cluster endpoint
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job s
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMEN
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

8	Phase 3	Streamlit Tab 5 (Health) Dashboard	Optional FastAPI; /predict-clust
9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
10	Phase 4	Railway deployment + scheduled jobs	Deployed live on Railway URL; retraining job s
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMEN
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

9	Phase 3	Local testing + polishing	streamlit run app.py works; no errors
---	---------	---------------------------	---------------------------------------

10	Phase 4	Railway deployment + scheduled retrain on Railway URL; retraining job	Dashboard live on Railway URL; retraining job
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

10	Phase 4	Railway deployment + scheduled retrain on Railway URL; retraining job	Dashboard live on Railway URL; retraining job
11	Phase 4	Documentation + demo video	README, METHODOLOGY.md, DEPLOYMENT
12	Phase 4	Final QA + client handoff	All exit gates met; ready for production

Freelance Pricing Guide

Deliverable	Scope	Price	Timeline
Full system (all 4 phases)	Complete RFM + KMeans + MLflow + Streamlit dashboard	\$8,000 - \$10,000	12 weeks
MVP: Backend only	RFM + KMeans + MLflow	\$3,000 - \$6,000	4 weeks
Streamlit dashboard	Dashboard on pre-built RFM data	\$2,500 - \$5,000	3 weeks
Custom data + retraining	Client provides transaction data as full pipeline	\$5,000 - \$12,000	5 weeks
Weekly monitoring + support	Scheduled retraining + dashboard	\$1,500 - \$3,000/mo	Ongoing

Full system (all 4 phases)	Complete RFM + KMeans + MLflow + Streamlit dashboard	\$8,000 - \$10,000	12 weeks
MVP: Backend only	RFM + KMeans + MLflow	\$3,000 - \$6,000	4 weeks
Streamlit dashboard	Dashboard on pre-built RFM data	\$2,500 - \$5,000	3 weeks
Custom data + retraining	Client provides transaction data as full pipeline	\$5,000 - \$12,000	5 weeks
Weekly monitoring + support	Scheduled retraining + dashboard	\$1,500 - \$3,000/mo	Ongoing

Full system (all 4 phases)	Complete RFM + KMeans + MLflow + Streamlit dashboard	\$8,000 - \$10,000	12 weeks
MVP: Backend only	RFM + KMeans + MLflow	\$3,000 - \$6,000	4 weeks
Streamlit dashboard	Dashboard on pre-built RFM data	\$2,500 - \$5,000	3 weeks
Custom data + retraining	Client provides transaction data as full pipeline	\$5,000 - \$12,000	5 weeks
Weekly monitoring + support	Scheduled retraining + dashboard	\$1,500 - \$3,000/mo	Ongoing

MVP: Backend only	RFM + KMeans + MLflow + Streamlit dashboard	\$3,000 - \$6,000	4 weeks
Streamlit dashboard	Dashboard on pre-built RFM data	\$2,500 - \$5,000	3 weeks
Custom data + retraining	Client provides transaction data as full pipeline	\$5,000 - \$12,000	5 weeks
Weekly monitoring + support	Scheduled retraining + dashboard	\$1,500 - \$3,000/mo	Ongoing

Streamlit dashboard	Dashboard on pre-built RFM data	\$2,500 - \$5,000	3 weeks
Custom data + retraining	Client provides transaction data as full pipeline	\$5,000 - \$12,000	5 weeks
Weekly monitoring + support	Scheduled retraining + dashboard	\$1,500 - \$3,000/mo	Ongoing

APPENDIX A: DATASET COMPARISON TABLE

Dataset	Records	Customers	Features	Ease	Recommended Use
UCI Online Retail	500k	5k	8	Easy (clean)	**Primary choice** — r
Synthetic (Faker)	50k	5k	8	Easy (generated)	Demo; portfolio MVP
Supermarket (UCI)	5k	1k	4	Easy (small)	Quick prototype
Kaggle E-Comm	Varies	Varies	Varies	Medium	Alternative; varies by d
UCI Online Retail	500k	5k	8	Easy (clean)	**Primary choice**
Synthetic (Faker)	50k	5k	8	Easy (generated)	Demo; portfolio MVP
Supermarket (UCI)	5k	1k	4	Easy (small)	Quick prototype
Kaggle E-Comm	Varies	Varies	Varies	Medium	Alternative; varies by d
UCI Online Retail	500k	5k	8	Easy (clean)	**Primary choice**
Synthetic (Faker)	50k	5k	8	Easy (generated)	Demo; portfolio MVP
Supermarket (UCI)	5k	1k	4	Easy (small)	Quick prototype
Kaggle E-Comm	Varies	Varies	Varies	Medium	Alternative; varies by d
Synthetic (Faker)	50k	5k	8	Easy (generated)	Demo; portfolio MV
Supermarket (UCI)	5k	1k	4	Easy (small)	Quick prototype
Kaggle E-Comm	Varies	Varies	Varies	Medium	Alternative; varies by d

APPENDIX B: REPRESENTATIVE RFM DISTRIBUTIONS (From UCI Online Retail)

Recency: min=0, median=70, mean=92, max=374 (days)
Frequency: min=1, median=5, mean=9, max=209 (purchases)
Monetary: min=0.5, median=205, mean=1890, max=280,000 (GBP)

Recency: min=0, median=70, mean=92, max=374 (days)

Frequency: min=1, median=5, mean=9, max=209 (purchases)

Monetary: min=0.5, median=205, mean=1890, max=280,000 (GBP)

Distribution is ****non-normal, right-skewed**** — typical for e-commerce (Pareto distribution: few high-value custom

Distribution is **non-normal, right-skewed** — typical for e-commerce (Pareto distribution: few high-value customers, many low-value). This is why Silhouette > 0.5 is realistic but challenging.

CONCLUSION

CustomerSegment AI is a **production-grade unsupervised learning project** that demonstrates:

- RFM engineering** — a non-trivial feature transformation widely used in real business ML
- Rigorous cluster validation** — silhouette optimization, not just "run KMeans once"
- Business translation** — personas that stakeholders can act on
- Modern tech stack** — Streamlit, MLflow, DVC
- Portfolio credibility** — real dataset, quantified metrics, clear methodology

Timeline: 12 weeks (half the complexity of LeadForge).

Dataset: UCI Online Retail (5k customers, 500k transactions).

Dashboard: Streamlit (5 tabs, interactive, deployment-ready).

CustomerSegment AI | Customer RFM Clustering Platform | v1.0 | Production-ready. Portfolio-grade. Stakeholder-approved.